DeepL Proに登録すると、より大きなサイズの文書ファイルを翻訳できます。
詳しくは、www.DeepL.com/pro をご覧ください。

GIM6010-8 マイクロモータ 取扱説明書

カタログ

バージョン記録 **2**

備考 **3**

法的通知 **3**

アフターセールス・ポリシー **3**

1 モーター仕様 **5**

1.1 図面と寸法 5

1.2 電気的特性 6

1.3 機械的特性 7

2 ドライブ情報 **8**

2.1 外観と3D寸法 8

2.2 インターフェースの概要 9

2.3 仕様 9

2.4 インターフェース詳細定義 10

2.5 主要コンポーネントと仕様 14

3 チューニングの説明 **15**

3.1 ユーザーガイド 15

3.2 ファームウェアアップデートダウンロード 40

4 通信プロトコルと例 **42**

4.1 CAN SIMPLEプロトコル ..... 42 CANOPENプロトコル

4.2 CANOPENプロトコル 55

4.3 RS485プロトコル（MODBUS RTU） 58

4.4 SDK 81

5 よくある質問と例外コード（更新予定） **92**

5.1 よくある質問（FAQ） 92

5.2 例外コード 92

バージョン記録

バージョン番号	日付	改訂/説明
0.1	2023.12.5	odrivetool サポートの最初のバージョン
0.2	2023.12.15	コマンドリストとコマンドカテゴリーを追加
0.3	2023.12.19	Python、Arduino、ROS SDKの追加
0.4	2023.12.23	USBおよびCAN互換スイッチの説明を追加
0.5	2023.12.29	ホスト・コンピュータのチューニングとテストの実例を追加
0.6	2024.1.2	CANプロトコルとPythonチューニングライブ例の追加
0.7	2024.1.8	CANインターフェイスの120Rマッチング抵抗スイッチオプションを追加
0.8	2024.2.15	2番目のエンコーダ関連コンテンツとユーザーゼロ点設定の追加
0.9	2024.2.23	パラメータを変更し、インターフェース機能を呼び出すためのCANコマンドの追加
0.91	2024.3.12	ナショナル・ダウンロード・ソフトウェアのためのドライバー・ダウンロード・アドレスを追加
0.92	2024.3.22	CANのデフォルトボーレートと、2番目のエンコーダの初期ロータ位置範囲の説明を追加。 および2つ目のエンコーダの初期ロータ位置範囲の説明を追加。
1.0	2024.3.26	モーター温度保護に関する記述を追加
1.1	2024.5.20	デバッグガイドの追加
1.2	2024.5.24	CAN MITプロトコルのバグを修正
1.3	2024.7.2	エラーコード表の追加
1.4	2024.8.18	IDの修正方法の追加
1.5	2024.8.21	ブレーキ抵抗器の使用方法を追加
1.6	2024.8.31	異なるバージョン用の endpoints.json ファイルのダウンロードを追加
1.7	2024.9.5	CANOpenプロトコルの説明を追加
1.8	2024.10.8	例外タイプを統合し、エンコーダ例外コードのリストを追加
1.9	2024.10.15	ホストコンピューター「モーターウィザード」の操作方法の追加
2.0	2024.12.6	Modbus RTU プロトコルと C/C++ SDK の説明を追加。

注記

1. 本取扱説明書の操作パラメータに従ってください！
2. モータの運転中は、ドライバを損傷させないように、過電流と過電圧から電源を保護する対策をしてください。
3. 使用前に部品に異常がないことを確認してください。部品が不足していたり、破損している場合は、テクニカルサポートまでご連絡ください。
4. ドライブには逆接続防止機能はありません。電源を接続する前に2.4.1項を参照し、電源のプラスとマイナスが正しいことを確認してください。
5. 静電気による損傷を避けるため、ドライブの露出部に手で触れないでください！

法的注意事項

本製品をご使用になる前に、必ず本取扱説明書をよくお読みになり、内容に従って製品を操作してください。万一、取扱説明書の内容に従わずに本製品を使用し、物的損害や人身事故が発生した場合、当社は一切の責任を負いかねますのでご了承ください。本製品は多くの部品で構成されていますので、事故を避けるため、お子様を本製品に触れさせないでください。製品をより長くお使いいただくために、高温・高圧環境でのご使用はお避けください。本取扱説明書は、可能な限り多くの機能および指示を含むように印刷されています。しかし、製品の継続的な機能向上や設計変更などにより、お客様が購入された製品との間に不一致が生じる場合があります。

本取扱説明書は、色、外観など実際の製品と異なる場合がありますので、実際の製品をご参照ください。当社は、誤植、不正確な最新情報、手順および/または機器の改良のために、いつでも予告なく本マニュアルに必要な改良および変更を加えることがあります。そのような変更は、本マニュアルの新しいバージョンにアップロードされます。すべてのイラストは機能説明のみを目的としています。

アフターサービス方針

本製品のアフターサービスは、「中華人民共和国消費者権益保護法」および「中華人民共和国製品品質法」に基づき、厳格に実施されます。

1. 保証期間と内容

    1) オンラインを通じて本製品を注文した利用者は、契約締結の翌日から起算して7日以内であれば、理由の如何を問わず本製品を返品することができる。返品に際しては、有効な購入証明書を提示し、請求書を返送するものとします。利用者は、返品された商品が元の品質および機能を維持し、良好な状態であること、商品および付属品の商標およびロゴが損なわれていないことを確認し、ギフトがある場合はそれも返却しなければならない。商品、人為的な破損、人為的な解体、箱の紛失、スペアパーツの紛失があった場合、返品はできません。商品の返品にかかる物流費用は、利用者が負担するものとします。利用者が物流費用を清算しない場合、実際に発生した金額は返金額から差し引かれます。支払われた金額は、返品商品の受領日から7日以内に利用者に返金されます。返金方法は、支払方法と同じです。正確な受領日は、銀行や支払機関などの要因により影響を受ける場合があります。

    2) 利用者が製品にサインした翌日から7日以内に、人為的でない破損や性能上の不具合があり、当社のアフターサービスセンターが製品をテストして確認した場合、製品は利用者に返送され、利用者は有効な購入証明書を提示し、請求書を返送する必要があります。無料プレゼントがある場合は、それも返却しなければなりません。

    3) 利用者がサインをした翌日から7〜15日以内に、人為的でない破損が原因で製品に不具合が生じた場合、アフターサービスセンターによる確認の後、利用者は製品一式の交換を受けることができます。交換後、製品の保証期間は再計算されます。

4) 製品に欠陥がある場合、ユーザーが製品を受領し、弊社サービスセンターが確認した日から15日以降365日以内であれば、無償で修理サービスを受けることができます。欠陥製品の交換はAUOの所有物となります。製品に欠陥がない場合は、元の状態で返送されます。製品に欠陥がない場合、AUOのサービスセンターが製品を検査・確認した後、返品・交換をお断りすることがあります。

本取扱説明書のアフターサービス規定と当店のアフターサービス規定に矛盾がある場合は、当店のアフターサービス規定が優先するものとします。

2. 非保証規定

以下の場合は保証の対象外となります：

1) 保証期間内に故障した場合。

2) マニュアルの要求事項に従わない場合、誤った使用方法による製品の損傷。

3) 不適切な操作、メンテナンス、設置、変更、テスト、その他の不適切な使用による損傷。

4) 非品質故障に起因する通常の機械的消耗。

5) 落下、衝撃、液浸、激しい衝撃などを含むがこれに限定されない、通常とは異なる作業条件によって生じた損傷。

6) 天災地変（洪水、火災、落雷、地震など）または不可抗力に起因する損害。

7) ピークトルクを超える使用により生じた損害。

8) 当社の正規品でない場合、または購入したことを証明する法的な書類を提出しない場合。

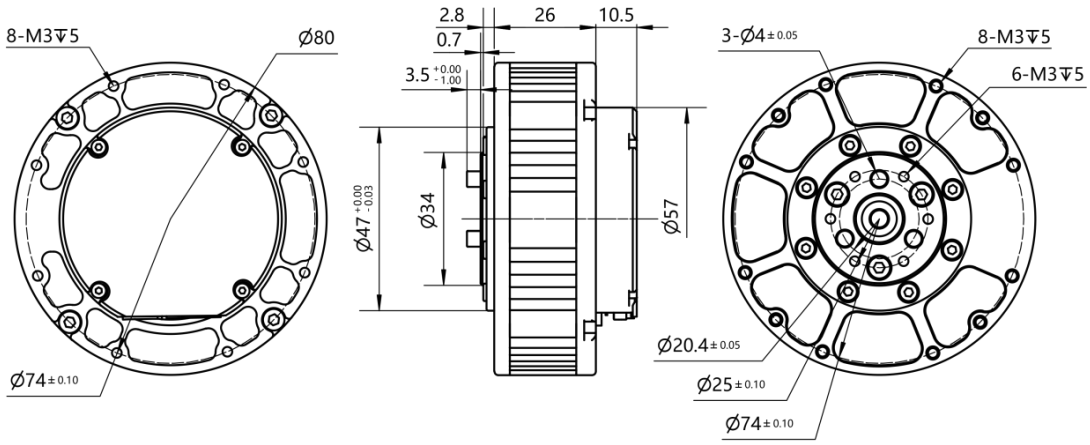
9) その他製品以外の設計、技術、製造、品質等に起因する故障または損傷。

10) 製品の私的な分解に起因する損害。

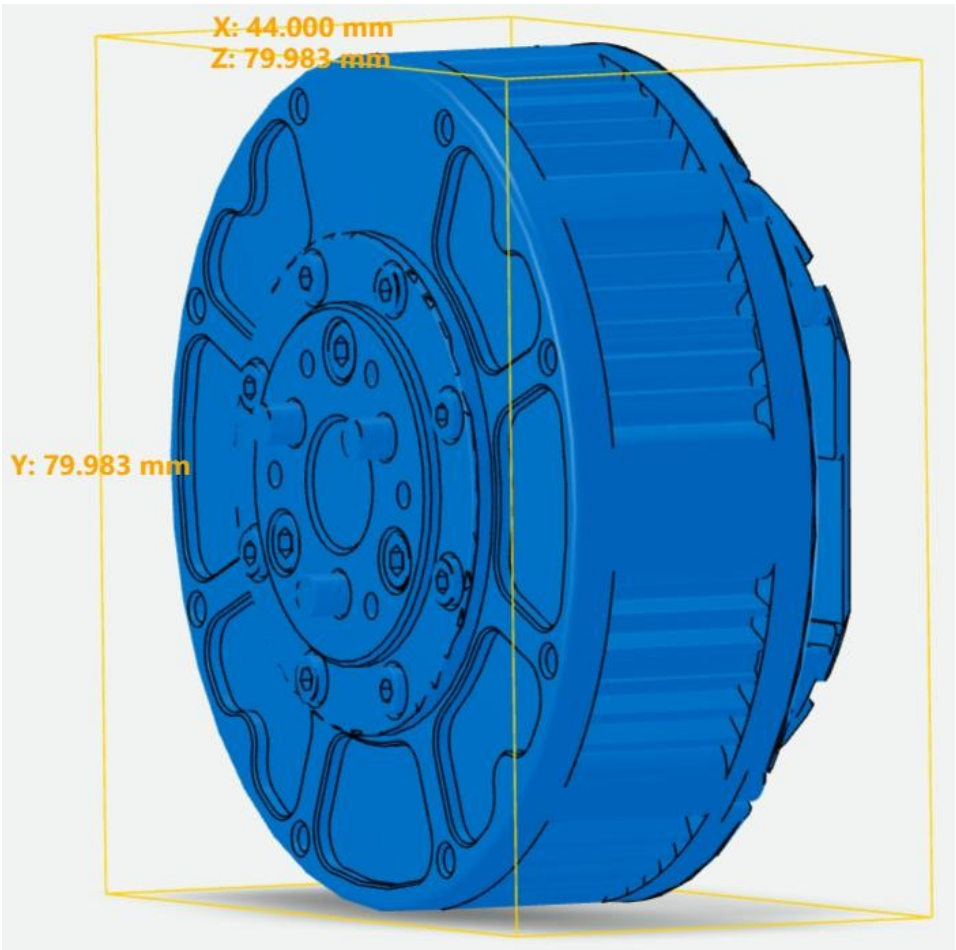
上記🔊の場合、ユーザーは費用を負担しなければならない。

1 モーター仕様パラメータ

1.1 図面と寸法



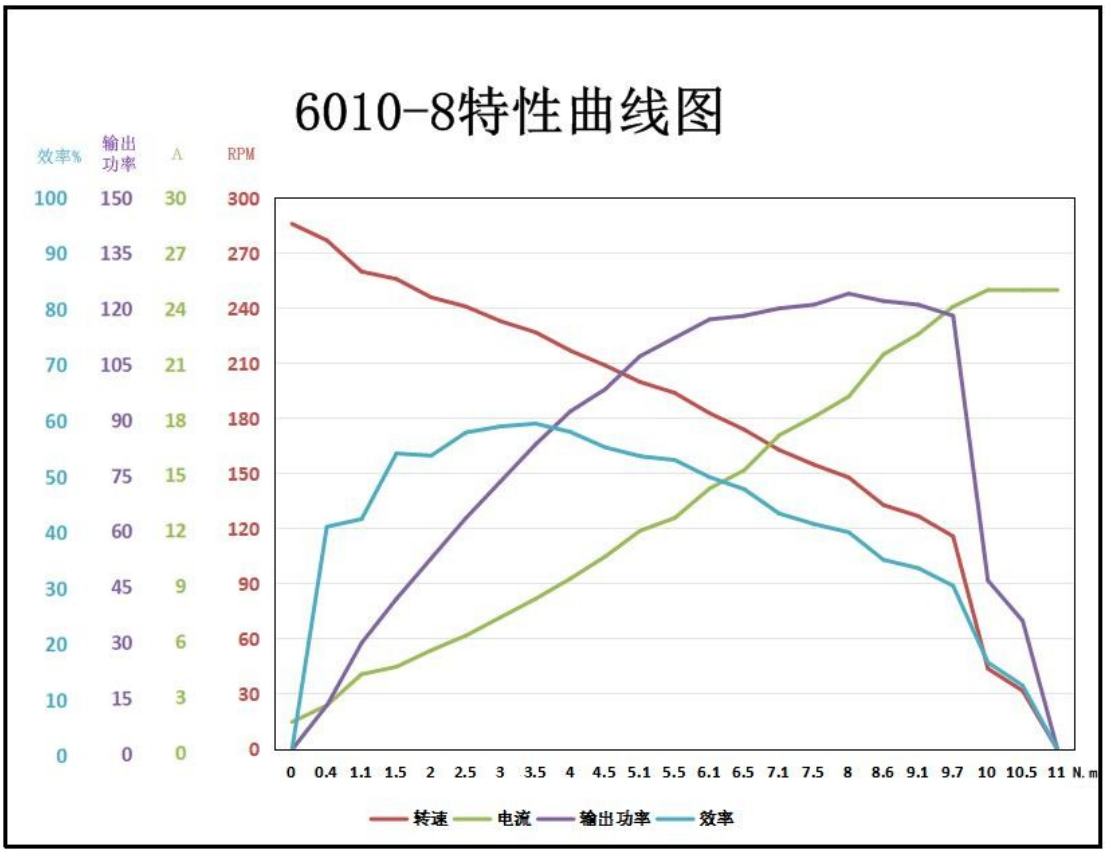
未注公差分段							
<=6	>6<=30	>30<=120	>120<=400	>400<=2000	比例	1 : 1	6010-8安装尺寸图
±0.1	±0.2	±0.3	±0.5	±1.0	重量kg	无	



1.2 電気特性

定格回転数	120rpm±10
最高回転数	420rpm±10
定格トルク	5N.m
トルクを妨げること	11N.m
評価される流れ	10.5A
プラグ電流	25A
無負荷電流	0.4A
絶縁抵抗/ステーター巻線	DC 500VAC、100Mオーム
高電圧抵抗/ステーターおよびハウジング	600VAC、1秒、2mA
モーター逆電位	0.054~0.057Vrms/rpm
位相間抵抗	0.48Ω±10%
相間インダクタンス	368μH±10
速度一定	12.3rpm/v
トルクの一定した	0.47N.M/A

特性曲線を以下に示す：

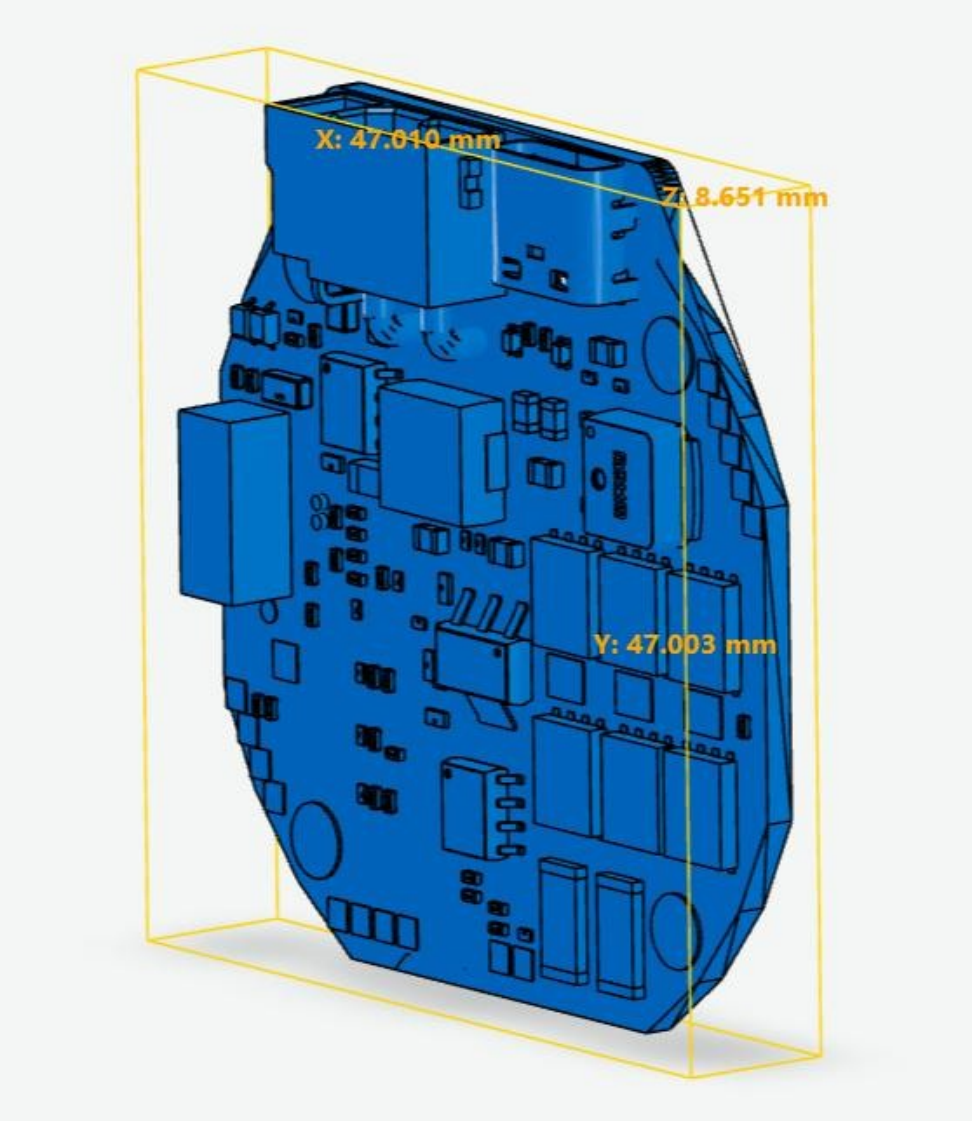


1.3 機械特性

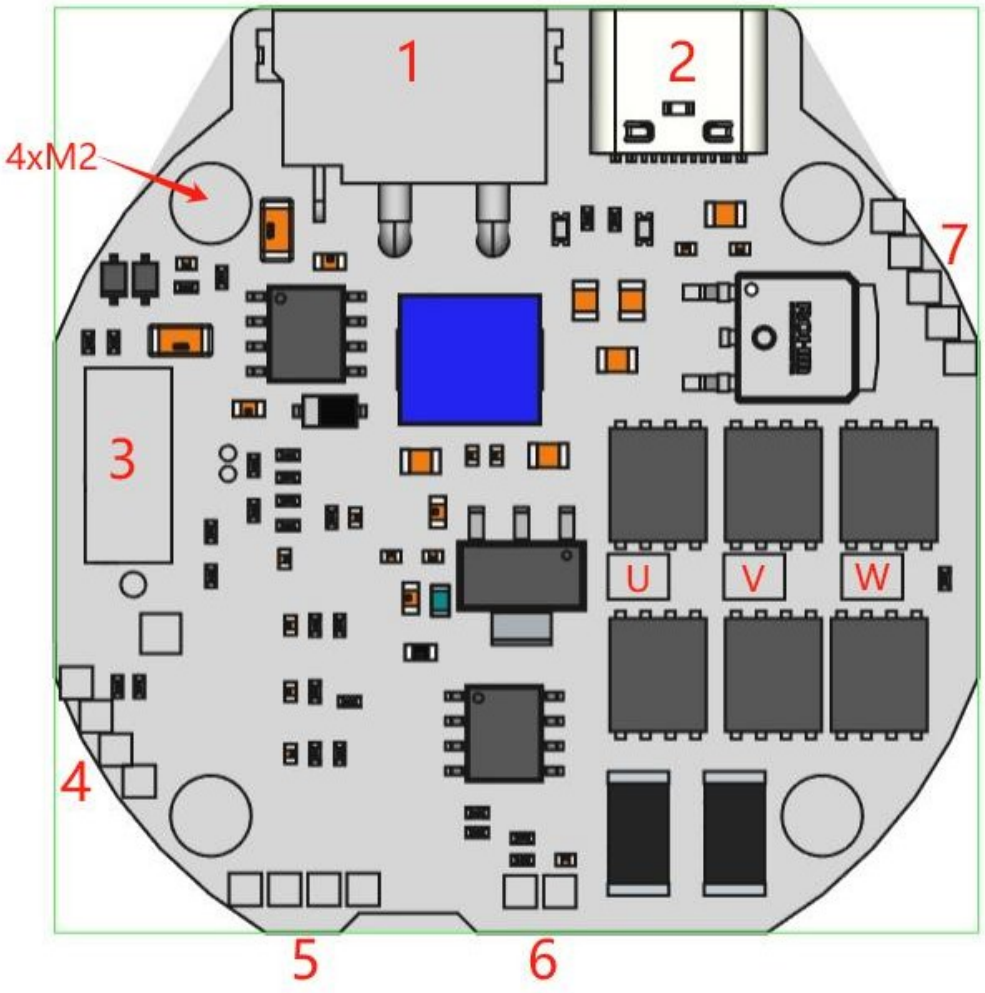
重量	388g±3
極数	14組
相数	3相
ドライバータイプ	FOC
減速比	8:1

2 駆動情報

2.1 外観と3D寸法



2.2 インターフェース概要



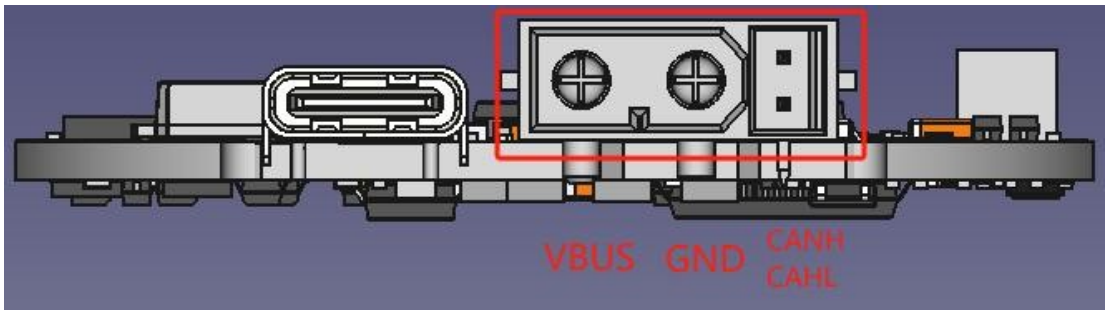
インターフェース番号	定義
1	15~60V電源およびCAN通信統合端子
2	Type-Cデバッグインターフェースおよびホストコンピュータ通信インターフェース
3	インターフェース拡張スロット（拡張可能RS485、EtherCAT、エアモデル、パルスオリエンテーション、オイル） (ドア制御用インターフェース／プロトコルなど)
4	SWDデバッグおよびダウンロード・インターフェース
5	第2エンコーダインターフェース（I2CおよびUARTサポート）
6	モーター温度インターフェース（NTC）
7	ブレーキ/ブレーキ抵抗インターフェース、12V電源、最小/最大リミットスイッチインターフェース
U/V/W	三相巻線はんだ付け穴
4xM2	取り付け穴

2.3 仕様

定格電圧	15~48V DC
最小/最大電圧	12/72V DC
定格電流	6A
最大ライン電流	30A
最大相電流	120A
待機時消費電力	<10mA
CANバス最大ボーレート	1Mbps
Type-Cレート	10Mbps
エンコーダー分解能	14ビット（1回転アブソリュート）
動作周囲温度	-20℃～70℃
アラームモータ温度	90℃（調整可能）
アラームドライブ基板温度	90℃（調整可能）

2.4 詳細インターフェース定義

2.4.1 電源およびCAN通信端子



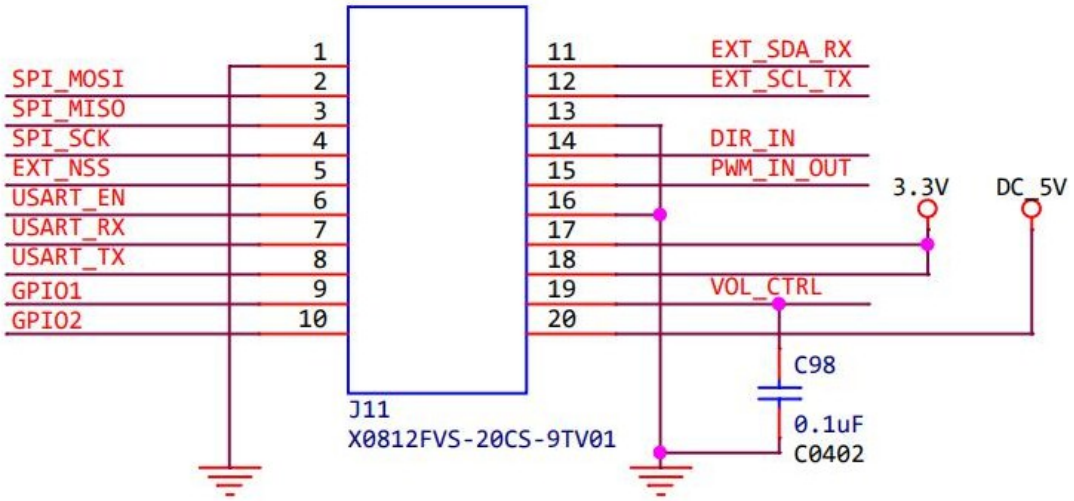
オンボード端子タイプXT30PB(2+2)-M、ワイヤーエンドタイプXT30(2+2)-F、ブランド名AMASS。RS485をサポートするバージョンでは、CANH/CANLはRS485 A/Bでもありますのでご注意ください。

2.4.2 Type-C デバッグインターフェース

Type-Cは標準ケーブル仕様を採用しており、一般的に使用されているPCや携帯電話のType-Cケーブルと互換性があります。

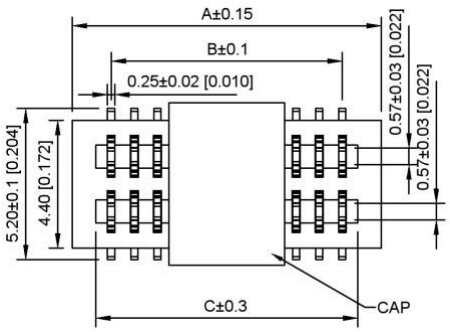
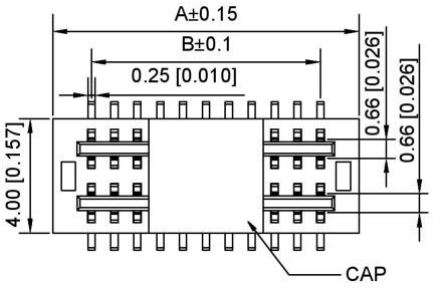
2.4.3 インターフェース拡張スロット

このスロットは、ボード間の豊富な拡張インターフェイスを提供し、サードパーティが任意の拡張ボードを開発できるように、以下のように設計されています。ハードウェアのバージョン3.9以降では、この拡張インターフェースは提供されませんのでご注意ください。



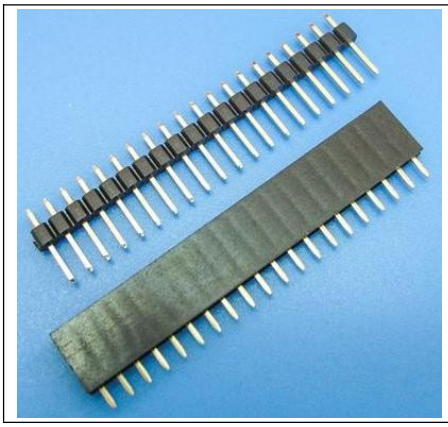
サードパーティは、SPI、USART、I2C、PWM、ADC、GPIOなどを介してドライバと対話し、様々な拡張機能を実現することができます。

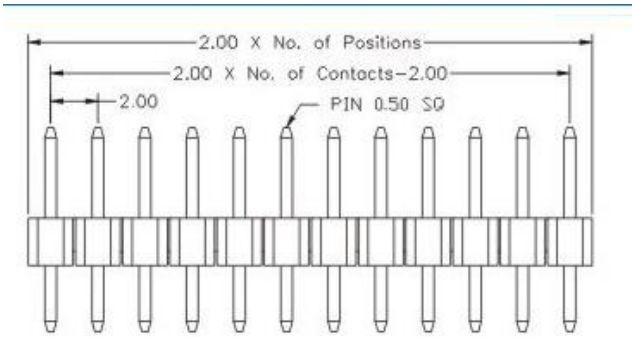
オンボード・スロットの型番はX0812FVS-20CS-9TV01（メス・シャーシ）、拡張ボード・スロットの型番はX0812WVS-20AS-9TV01（オス・シャーシ）で、興國のブランドである。

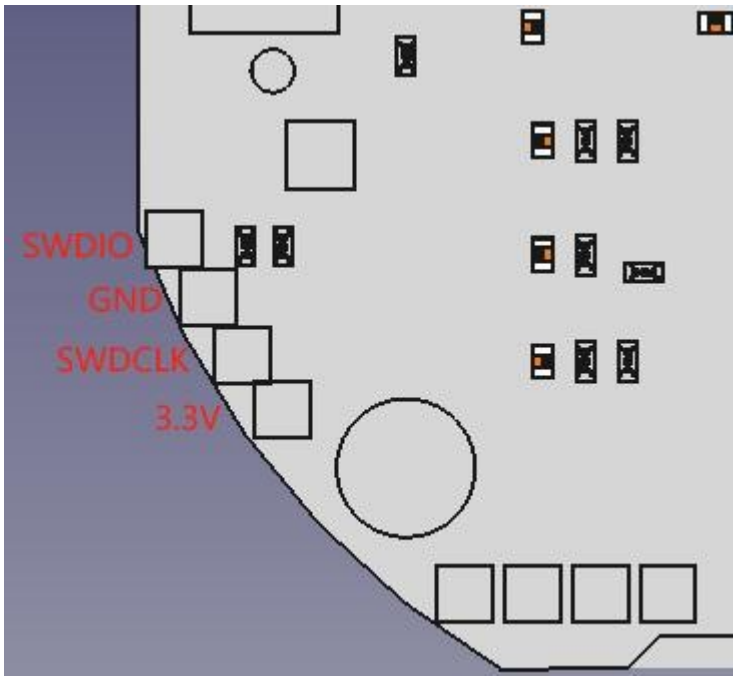
メス型X0812FVS-20CS-9TV01	オス X0812WVS-20AS-9TV01
	

2.4.4 SWD委託インターフェース

2mmピンホール、ユーザーは以下のように2mmストレート単列ピンをハンダ付けすることができます：



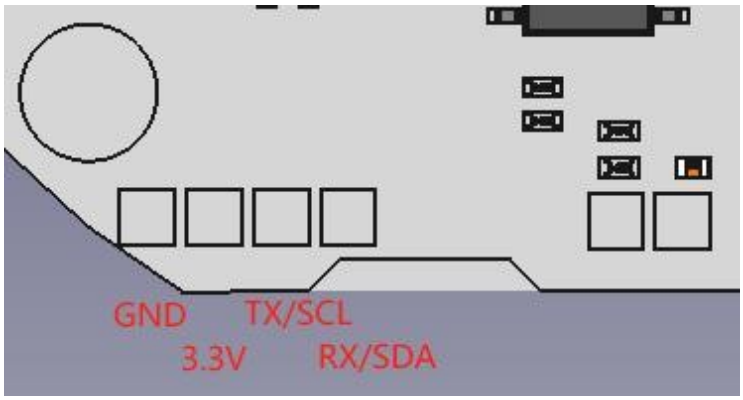




2.4.5 セカンドエンコーダインタフェース

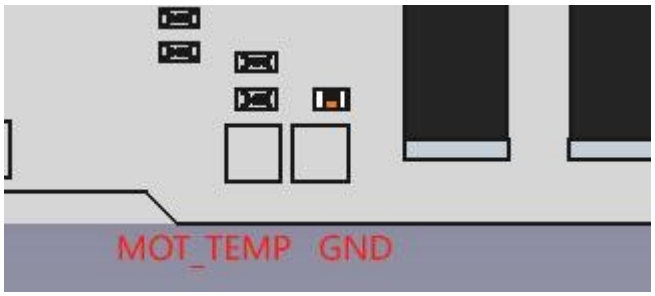
2mm間隔のピンホール、2mmストレート単列ピンをはんだ付け可能、2.4.4参照。

このインターフェースは、USART (TX/RX)またはI2C (SCL/SDA)を介して第2エンコーダと通信できます。



2.4.6 モーター温度インターフェース

モータには10K NTC抵抗が内蔵されており、2本のリード線がMOT_TEMPとGNDにはんだ付けされている。

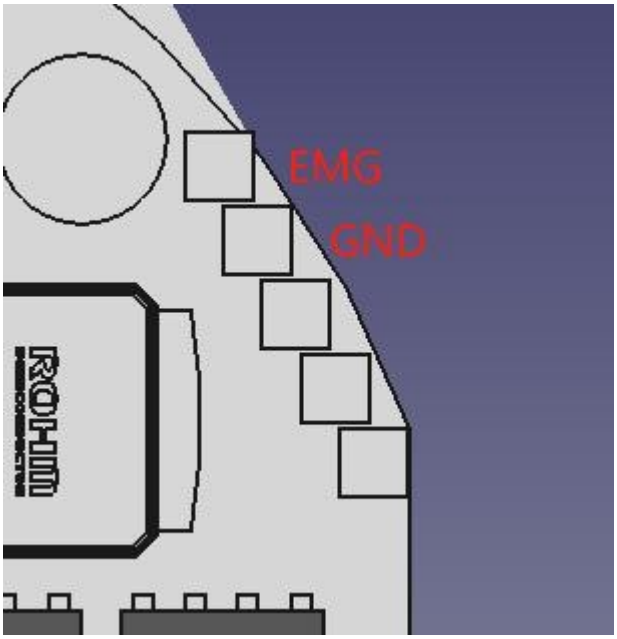


2.4.7 ブレーキ/保持抵抗コネクタ

図に示す5ピンコネクタの上部2つのはんだ穴は、ブレーキ/ブレーキ抵抗インターフェース用で、ピン穴間は2mm で、ユーザーは2mm ストレートの単列ピンをはんだ付けすることができます。

ブレーキインターフェイスの場合、ドライバは電源投入時にこのインターフェイスに🔑電流を流し続け、ブレーキが開き、モータが正常に動作できるようにします。

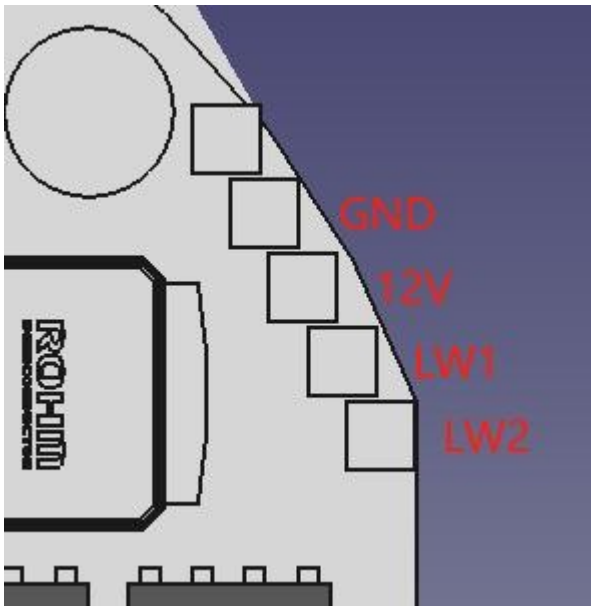
ブレーキ抵抗インターフェイスを使用する場合、外付けのブレーキ抵抗（またはブリーダー抵抗）を接続することで、反 応電位がしきい値電圧より高いときにブレーキ抵抗を通して電流をブリードすることができ、非常ブレーキが利かなくなったり、反 応電位がドライブにダメージを与えたりすることを防ぐことができます。



2.4.8 リミットスイッチ・インターフェイス

ドライブは、2mm間隔のピン・ホール、ユーザーによるはんだ付けが可能な2mmストレートの単列ピンを持つ外部リ ミット・スイッチ用に、2つのリミット・スイッチ・インターフェースと12V電源を提供します（2.4.4参照）。

LW1は最小位置リミットスイッチ、LW2は最大位置リミットスイッチです。外付けの2線式または3線式のNPNスイッチを使用できます。



2.5 主要部品と仕様

No.	構成部品	型式/仕様	数量
1	マイコン	N32G455	1
2	ドライバ	FD6288Q	1
3	磁気エンコーダディスク	MA732、14ビットアブソリュート	1
4	MOSFET	JMSH1004NG、100V/120A	6

3 チューニング手順

3.1 スタートガイド

3.1.1 準備

モーターを作動させるには、以下のものが必要です：

✓ 電源

電源電圧の要件については第1章を参照してください。安定化電源またはバッテリーを推奨します。ユーザーからよく聞かれる質問は、電源の選び方です。以下は、参考までにいくつかの簡単な提案です：

電源選びのポイント

◆ 電流要件

一般的に、必要な電流は少なくとも5Aであるべきです。正確な値は、システムの電力要件と電圧によって異なります。

◆ 電圧要件

電圧要件は2つの要因に依存する：モータのKvとシステムで必要とされる最大速度RPMmax、および最大必要供給電圧の計算式：

$$_{a} = \frac{P_a}{} \times \ 1.2$$

ここで1.25は、システムの安全な電圧しきい値を与える経験的な係数です。

◆ 電力要件

電力については、単純に最高速度での最大電流値Imaxに依存します：

$$P_{a} = {}_{a} \times \frac{P_a}{} \times \ 1.2$$

✓ 電源＋通信インターフェースケーブル

6010-8には2+2電源通信ソケットが付属しています。2.4.1を参照し、電源のプラス端子とマイナス端子を正しく接続してください。

2.4.1を参照し、電源のプラス端子とマイナス端子を正しく接続してください。そうしないと、ドライブは逆接続に対して保護されていないため、ドライブを焼損する恐れがあります。一方、CANH/CANLなど2つの通信ラインの定義順序に注意してください。間違った接続は通信の誤動作につながります。

警告

- ✧ 特に乾燥した地域や乾燥した季節では、静電気によるドライブのインターフェイス・コアの損傷を防ぐため、通信バスに手で触れないようにしてください。
  特に乾燥した地域や乾燥した季節にはご注意ください！
- ✧ 電源端子を電気的に抜かないようにしてください！
- ✧ ドライブの電源エレメントを損傷する恐れがあるため、電源スイッチとしてオープン・サーキット・ブレーカーを使用しないでください！
- ✧ 72Vを超えないようにしてください！

✓ Type-C ケーブル

テストの初期段階では、USB Type-Cケーブルを使用してモーターをテストすることを強く推奨します。充電専用のType-Cケーブルではなく、最も一般的に使用されている携帯電話用のType-Cケーブルが使用できます。

Type-Cケーブルは、モーターはもちろん、ドライバーにも電力を供給しないことにご注意ください！

✓ 電源の入れ方

最初に電源を切り、電源ケーブルを接続してから電源を入れてください。電気でプラグを抜いたり、エアスイッチでプラスまたはマイナスのケーブルを切り替えて使用したりしないでください。

USB Type-Cケーブルは電源投入後、いつでも抜き差しできる。

3.1.2 上部ユニットを使用して

◆ モータウオド

1. モーターウィザードをインストールするには

、Windows用のインストールファイル（）をダウンロー

ドし、インストール手順に従ってください。

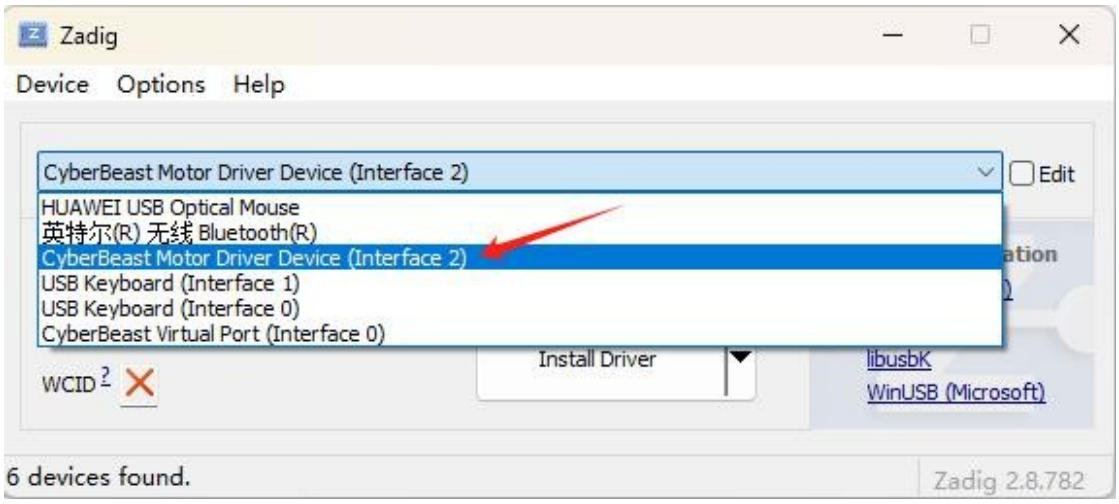
(https://bl.cyberbeast.cn/actuator/steadywin_motorwizard.exe) をダウンロードし、デフォルトのインストール手順に従ってください。

USBドライバをインストールします。

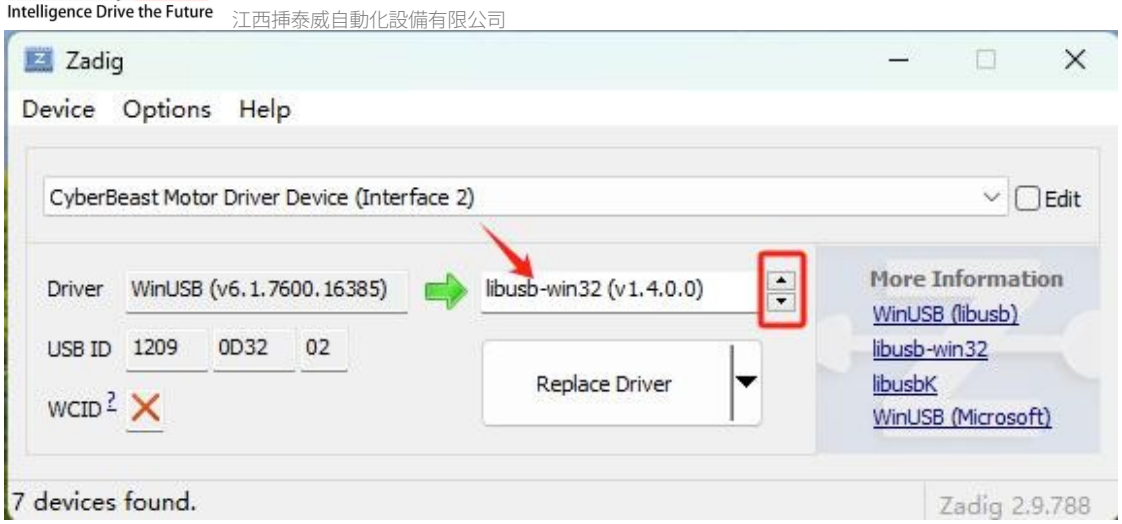
2. USB ドライバのインストール

https://zadig.akeo.ie にアクセスし、USBドライバツールZadigをダウンロードします。ドライブをType-CケーブルでPCに接続し、ドライブの電源ランプが点灯したら、Zadigを開き、ドロップダウンボックスから「CyberBeast Motor Driver Device (Interface 2)」を選択します。

（ドロップダウンリストから "CyberBeast Motor Driver Device (Interface 2) "を選択します：



上下のボタンをクリックして別のUSBドライバを選択し、このインターフェースの"libusb"ドライババージョンを選択して、このインターフェースの"Install Driver"または "Replace Driver"をクリックしてください。ドライバ "をクリックしてこのインターフェースのドライバをインストールしてください：



odrivetool

このドライバはodrive (https://github.com/odriverobotics/odrive.git)と互換性があるので、odrivetoolをテスト用のホストコンピュータとして使用することができます。

以下の手順に従ってodrivetoolをインストールしてください：

➢ **Windows**

1. pythonのインストール

pythonの公式サイトhttps://www.python.org、最新のpythonインストーラーをダウンロードし、指示に従ってください。サードパーティのウェブサイトやマイクロソフトのマイクロソフトストアからpythonのバージョンをダウンロードしないでください。

2. ビジュアルc++ビルダーのインストール

visual c++ ジェネレーター・ツール https://visualstudio.microsoft.com/visual-cpp-build-tools/ をインストールし、下図に示すように、インストール・プロセス中に「Desktop development with C++」にチェックを入れる。



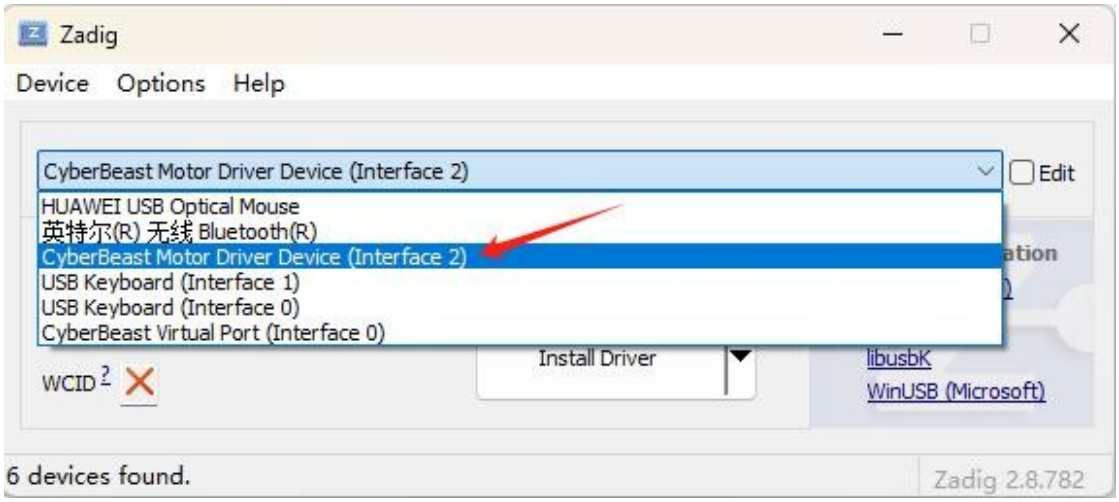
3. odrivetoolのインストール

Windows PowerShellを管理者として実行し、そこからpip install odriveを実行し、Enterしてインストールする。インストールの途中でエラーが表示された場合は、再度インストールしてください。また エラーが出た場合は、コンピュータを再起動してやり直してください。

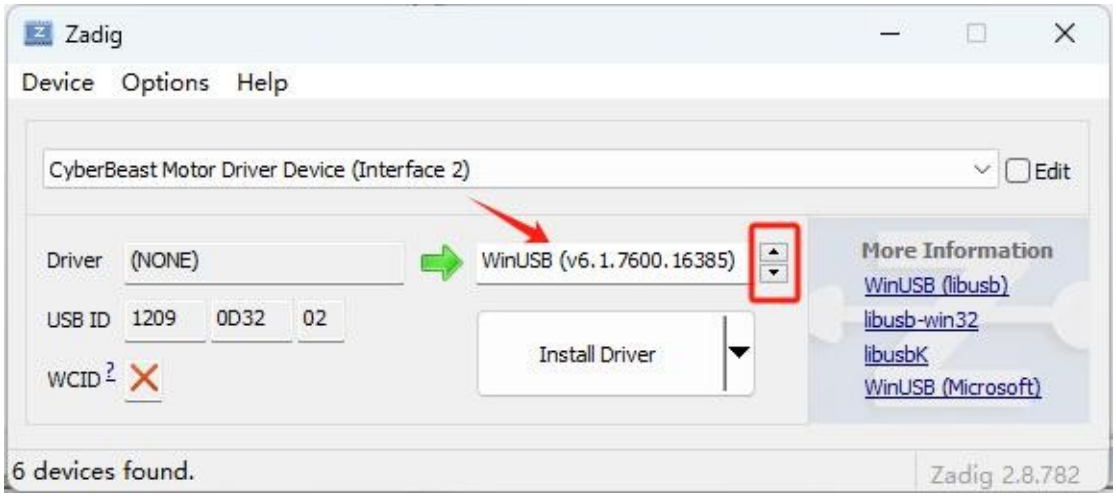
4. USBドライバのインストール

https://zadig.akeo.ie にアクセスし、USBドライバツールZadigをダウンロードします。ドライブをType-Cケーブルでコンピュータに接続し、ドライブの電源ランプが点灯したら、Zadigを開き、ドロップダウンボックスから「CyberBeast Motor Driver Device (Interface 2)」を選択します。

（ドロップダウンリストから "CyberBeast Motor Driver Device (Interface 2) "を選択します：



上下キーをクリックして別のUSBドライバを選択し、このインターフェースの「WinUSB」ドライババージョンを選択して、「ドライバのインストール」をクリックして、このインターフェースのドライバをインストールしてください：



➢ **WSL (Windows Subsystem for Linux)**

1) python/usb/odrivetoolのインストール

WSL2がインストールされている場合は、WSL2のコマンドラインに移動し、以下の手順に従ってください（WSL2があると仮定しています）。

```
sudo apt install python3 python3-pip sudo apt install libusb-
1.0-0
sudo pip install odrive numpy matplotlib
```

(Ubuntuがインストールされている）：

1行目のコマンドはpythonをインストールし、2行目のコマンドはusbドライバをインストールし、3行目のコマンドはodrivetool uplinkをインストールします。

2) ドライブをWSLに接続する

デフォルトでは、WindowsはこのUSBポート用のドライバをロードしますが、WSL用のドライバはロードしません。このUSBポートをWSLにロードするには、マイクロソフトのドキュメント

(https://learn.microsoft.com/zh-cn/windows/wsl/connect-usb）を参照してください。

➢ **Ubuntu**

Ubuntu でのインストール手順は WSL でのインストール手順とよく似ています。

3.1.3 モーターパラメータの設定

警告

このセクションは、正常に動作させ、モーターの焼損を避けるために重要ですので、注意深くお読みになることをお勧めします！

前のセクションで説明したようにホストコンピュータ（Motor Wizardまたはodrivetool）を正常にインストールした後、モーターに電源を入れ、USB Type-Cケーブルを接続します：

- モータウィザード



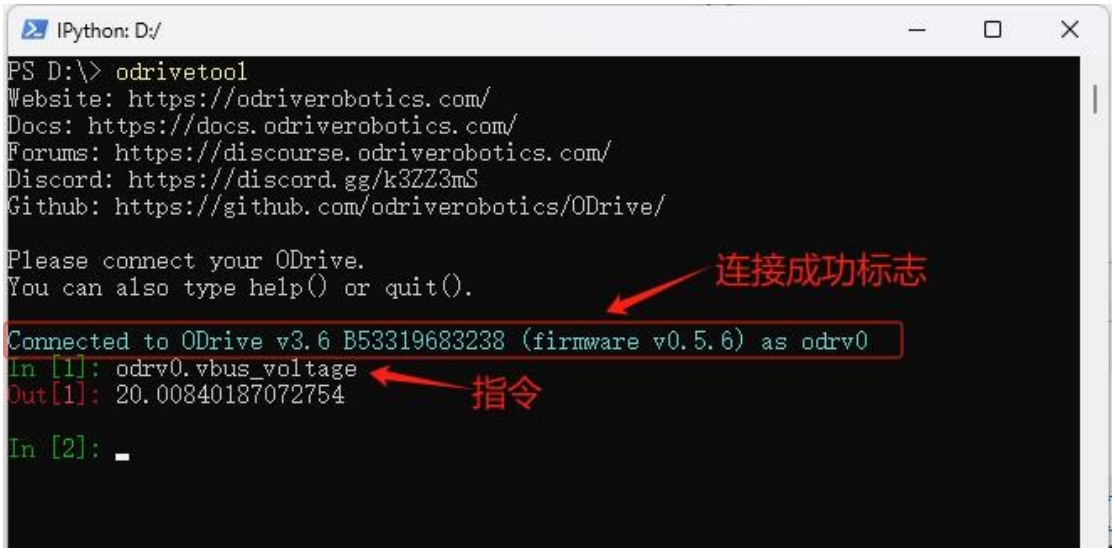


モーターウィザードを開きます。接続に成功すると、上図（左）のようにモーターのステータスと電圧が表示されます。

Motor Parameters "をクリックすると、ドライブのハードウェア・バージョンとファームウェア・バージョンが画面上部に表示されます。上図右のように、ハードウェア・バージョンは3.8.3、ファームウェア・バージョンは0.5.13です。

◆ オドリベツール

シェル(Windows PowerShellまたはLinux Terminal)でodrivetoolを実行します(odrivetoolと入力してエンター)：



コマンド例：odrv0.axis0.controller.input_vel、odrv0は現在接続されているモーターを表し、デフォルトでは、最初に接続されているモーターはodrv0と呼ばれ、2番目はodrv1と呼ばれるなど、axis0はドライブに接続されている最初のモーターを表し、現在のバージョンでは1つのモーターだけがドライブに接続されています。この命令の意味は、ドライブの現在の速度制御目標値を問い合わせることです。

操作のヒント
◆ TABキーを頻繁に使用すると、Linuxのコマンドプロンプトに似たコマンドプロンプトが表示されます。 コマンド；
◆ 上下キーはコマンドの履歴を表示します。
◆ コマンドを入力すると、似たようなコマンドの履歴が表示され、右ボタンで履歴が完了する。

➢ **キーしきい値（リミット）の設定**

◆ モタウザド

モーターウィザードの "Motor Parameters "をクリックすると、下図の赤枠のように、電圧範囲、電流しきい値（最大電流）、速度しきい値（最大速度）、校正電流のキーしきい値パラメータを設定することができます。



電流しきい値は温度係数にも影響されますので、次のサブセクション（odrivetool）の説明を参照してください。

速度しきい値は通常、速度制御または位置制御モードでの制限ですが、トルク制御で速度制限をオンにすることも可能です。このスイッチはモーターウィザードの "Drive Parameters "をクリックすることでオンにすることができます：



◆ odrivetool

```
odrv0.axis0.motor.config.current_lim= 30
```

● 電流しきい値

上記のコマンドは、電流しきい値を 30 A に設定します。この電流しきい値は、供給電流ではなく Q 軸電流を指していることに注意してください。このしきい値は、出力🔑トルクを直接制限します。6010-8では、このしきい値を50A以上に設定しないでください！

電流しきい値に影響を与えるその他の要因
◆ モータ温度 モータ温度保護が有効になっている場合 (odorv0.axis0.motor.motor_thermistor.config.enabled=1)、モータの 電流温度もQ軸電流に影響します。
◆ ドライブ基板の温度 ドライブ基板の温度保護が有効になっている場合は (odorv0.axis0.motor.fet_thermistor.config.enabled=1)、ドライブ基板の現在 の温度も Q 軸電流に影響します。 基板の電流温度も Q 軸電流に影響する。
上記2つの温度の影響は非常に似ており、以下の式で表すことができます：

$$'_i = \frac{=}{-} \times \quad _i$$

ここで$'_i$ は最終的な有効電流しきい値、$_i$は設定された電流しきい値、 は電流温度（モーター温度または基板温度）、設定温度下限値（motor_thermistor.config.temp_limit_lower およびfet_thermistor.config.temp_limit_lower ）、設定温度上限値（motor_thermistor.config.temp_limit_lower）である。

(motor_thermistor.temp_limit_upperおよびfet_thermistor.config.)。

```
odrv0.axis0.controller.config.vel_limit= 30
```

- 速度しきい値

システムのグローバル速度制限で、上記のコマンドによって30turn/sに制限される。デフォルトでは、この速度制限は純粋なトルク・モードでは機能しないが、以下のスイッチをオンにすることで有効にできることに注意：

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit=1
```

- キャリブレーション電流

この電流のデフォルト値は5Aで、デフォルトでは変更する必要はありませんが、ユーザーの電源電流が小さい場合は、この値を減らすことができます！

```
odrv0.axis0.motor.config.calibration_current=2
```

そうすると、低電圧アラームが時間通りに表示されます。

➢ **主要ハードウェアパラメータの設定**

◆ モータウィザード

モーターウィザードの "Motor Parameters "をクリックします：



これらのパラメータの意味は次のサブセクション（odrivetool）で説明します。

◆ オドリベツール

- **最大放電/充電電流**

放電電流は電源からドライブとモータに供給される順方向電流で、充電電流は電源に流れ込む逆方向電流です。この2つの値は電源に関係しますので、電源が放電できずに電圧が引き下げられたり、逆起電力で電圧が下がったりしないよう、適切な値に設定してください。

```
odrv0.config.dc_max_negative_current odrv0.config.dc_max_positive_current
```

破損。ただし、これら2つの値の絶対値が小さい値に設定されていると、アラームが発生しやすいので注意。

- **極数ペア**

極ペア数は、モータ・ロータの極数を 2 で割った値です。較正を成功させるには、ユーザがこの値を正しく設定する必要があります。

```
になります。
```

にキャリブレーション・アラームが発生します。

- **トルク定数**

トルク定数は、モータによって生成されるトルクをQ軸電流で割ったもので、**⊠⊠によって**モータKv値に関連付けられます。＝

```
odrv0.axis0.motor.config.torque_constant
```

8.27/.

トルク定数が正しいかどうかはモーターの動作には影響しませんが、ユーザーがトルク制御用に入力した値の単位変換には影響します。 ユーザーがトルク制御用に Nm の代わりに単位 A を使用したい場合は、この値を 1 に設定するだけです。

- **温度感知**

ドライブボードとモータには NTC 温度センサが内蔵されており、有効にすると温度に応じて出力値が制御されます。

```
# モーター温度保護 odrv0.axis0.motor.motor_thermistor.config.enabled 1 =
odrv0.axis0.motor.motor_thermistor.config.temp_limit_lower= 20
odrv0.axis0.motor.motor_thermistor.config.temp_limit_upper= 100 # ドライブ基板の温度保護
odrv0.axis0.motor.fet_thermistor.config.enabled= 1
odrv0.axis0.motor.fet_thermistor.config.temp_limit_lower= 20
odrv0.axis0.motor.fet_thermistor.config.temp_limit_upper= 100 # 温度取得
odrv0.axis0.motor.motor_thermistor.temperature # モータ温度
odrv0.axis0.motor.fet_thermistor.temperature # ドライバ基板温度
```

電流（トルク）を取得し、ドライブとモーターを保護します。

- **PIDチューニング**

```
odrv0.axis0.controller.config.pos_gain=20.0 odrv0.axis0.controller.config.vel_gain=0.16
odrv0.axis0.controller.config.vel_integratorゲイン=0.32
```

以下の手順は、ユーザーが PID パラメータを調整する際の参考となります：

1. PIDの初期値を設定する。
2. vel_integrator_gain を 0に設定します。

```
odrv0.axis0.controller.config.vel_integrator_gain=0
```

3. vel_gain の方法を調整する：
   1) 速度制御モードでモーターを回転させ、回転が滑らかでない、ぎくしゃくしている、振動している場合は、回転が滑らかになるまでvel_gainを減らします。
   2) その後、目立つ揺れが発生するまで、vel_gainを毎回約30%ずつ増加させる。
   3) この時点で、安定するまでvel_gainを50%程度下げる。

4. pos_gainの調整方法：

   1) 位置モードでモーターを回転させてみて、回転がスムーズでなく、引っ張りや振動がある場合は、回転がスムーズになるまでpos_gainを下げる。

   2) その後、位置制御🔊が明らかなオーバーシュート（位置制御モーターが目標位置をオーバーシュートした後、目標位置に戻るたびに振動すること）を示すまで、pos_gainを毎回約30%ずつ増加させる。

   3) その後、オーバーシュート現象が消えるまでpos_gainを徐々に下げる。

5. 上記の4つのステップの後、vel_integrator_gainを0.5*bandwidth*vel_gain（bandwidthはシステム制御帯域幅）に設定できます。制御帯域幅とは？例えば、ユーザが目標位置を設定してからモータが実際に目標位置に到達するまでの時間が10msの場合、制御帯域幅は100Hzとなり、vel_integrator_gain=0.5*100*vel_gainとなります。

上記のチューニングプロセスでは、肉眼で認識される誤差を避けるために、3.1.8 のグラフィカルな手段を使用してチューニング効果をリアルタイムで確認することをお勧めします。モーターウィザードを使用する場合、PID調整効果は「見たものが得られるもの」であり、下図はPIDパラメータを示しています：



3.1.4 パワーオンキャリブレーション

初めてマイクロモータを使用するときは、モータとエンコーダのキャリブレーションが必要です。校正を行う前に、モーターを固定するか、手でしっかりと持ち、出力🔊軸が無負荷の状態で、校正プロセスは以下の通りです：

警告

較正の前に、モーターを無負荷の状態にしてモーターを固定してください！

◆ モタウザド

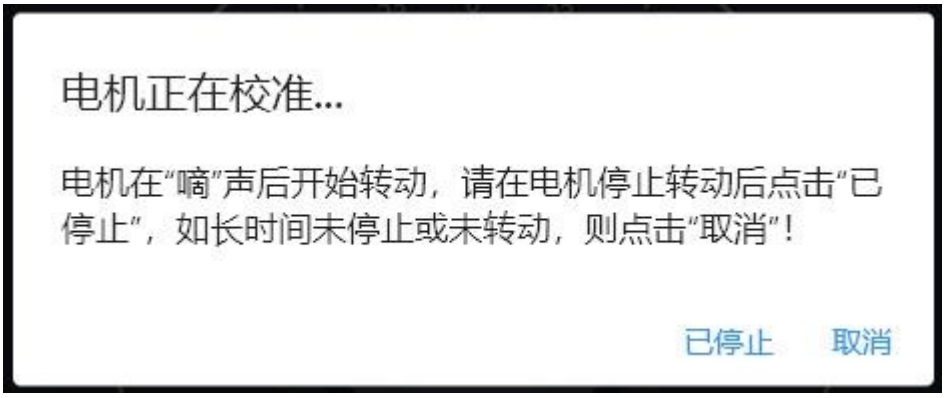
モーターウィザードを開き、"Start Motor "をクリックします。モーターが較正されていない場合、以下のようなプロンプトが表示されます：



モーターが較正されていない場合、以下のメッセージが表示されます。"OK "をクリックすると較正プロセスに進みます。または、メイン画面の "Motor Parameters "をクリックし、"Basic Parameters "にモータの較正状態が表示されます。

Basic Parameters "セクションに、モーターの較正ステータスが表示されます。未校准、ステータスの右側にある "Calibrate "ボタンをクリックして較正プロセスに入ります。

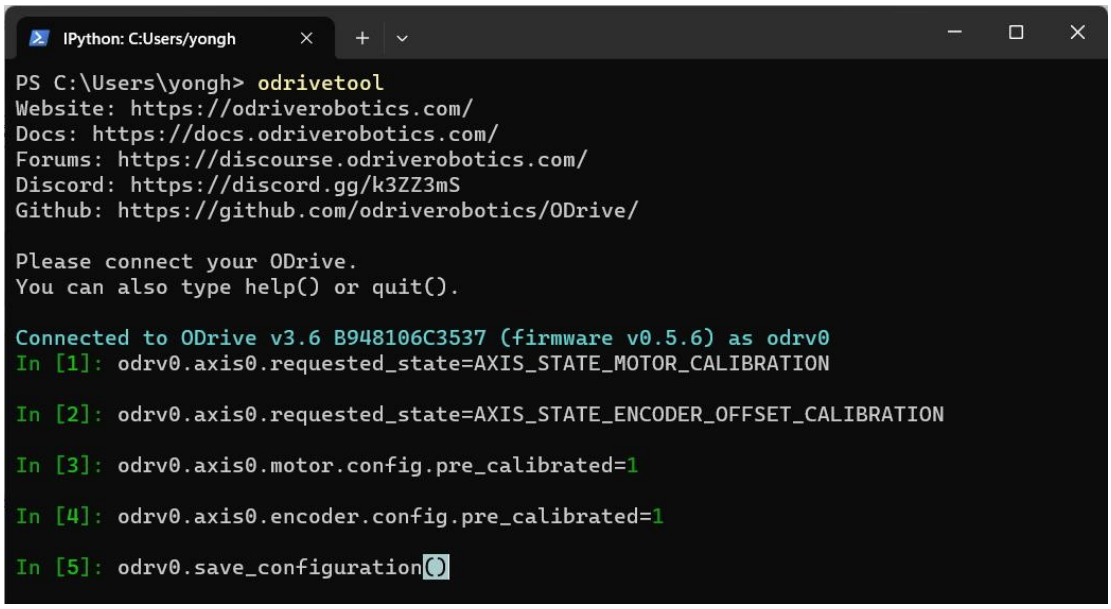
較正プロセスに入ると、画面にプロンプトが表示されます：



この時点で、モーターはビープ音を発し、回転し始めます。モータの回転を観察し、一定時間後に停止した場合は "Stopped "をクリックしてキャリブレーションプロセスを終了し、回転しない場合や5分後に停止した場合は "Cancel "をクリックしてください。

キャリブレーション終了後、ドライブが再起動しますので、1～3秒間お待ちください。この時点で "Motor Parameters "に戻り、已校准が表示されているか確認してください。

◆ オドリベツール




```
odrv0.axis0.requested_state=AXIS_STATE_MOTOR_CALIBRATION dump_errors(odrv0)
odrv0.axis0.requested_state = AXIS_STATE_ENCODER_OFFSET_CALIBRATION dump_errors(odrv0)
odrv0.axis0.motor.config.pre_calibrated=1
odrv0.axis0.encoder.config.pre_calibrated=1 odrv0.save_configuration()
```

ステップごとの説明は以下の通り：

➢ ステップ1: モータパラメータの自己識別

モータの相抵抗と相インダクタンスを測定すると、鋭い「ビープ音」が聞こえます。相抵抗と相インダクタンスの測定結果は、以下の表で確認できます。

```
odrv0.axis0.motor.config.phase_resistance
odrv0.axis0.motor.config.phase_inductance
```

上記のコマンドが表示されます：

➢ ステップ2：エラーコードの表示

もし赤色のエラーコードがあれば、モーターを再起動して再試行するか、アフターサービスに報告する必要があります。

➢ ステップ3：エンコーダの較正

エンコーダの取り付け角度とモーターの機械角度の較正、およびエンコーダ自体の較正を含むエンコーダの較正を行います。この較正プロセス中、モーターはゆっくりと1角度ずつ正回転し、その後1角度ずつ逆回転します。正回転だけで止まってしまう場合はエラーですので、ステップ4でエラーコードを確認してください。

➢ ステップ4：エラーコードを見る

ステップ3でエンコーダのキャリブレーションを行った後、システムのエラーコードを確認します。通常表示されるエラーはERROR_CPR_POLEPAIRS_MISMATCHで、これはエンコーダのCPRが正しく設定されていないか、モーターの極対番号が正しく設定されていないことを示します。

```
odrv0.axis0.encoder.config.cpr odrv0.axis0.motor.config.pole_pairs
```

以下のコマンドで確認・設定してください：

➢ ステップ5：モーターキャリブレーション成功フラグの書き込み

➢ ステップ6：エンコーダキャリブレーション成功フラグの書き込み

```
odrv0.axis0.encoder.config.pre_calibrated=1
odrv0.axis0.motor.config.pre_calibrated=1 odrv0.save_configuration()
```

➢ ステップ7：較正結果の保存と再起動

3.1.5 IDの変更

ドライブIDはバス固有のもので、範囲は0～63、デフォルトIDは0です。バス上に複数のモーターが直列に接続されている場合は、各モーターに異なるIDを設定する必要があります：

➢ モータウィザード ID の修正

Motor Wizardを開き、モータを接続して "Motor Parameters "をクリックし、"Communication Parameters "の列でIDを変更し、"Synchronise "をクリックしてドライバに保存します。



➢ USB IDの変更

```
odrv0.axis0.config.can.node_id= xxx
```

ユーザーがodrivetool経由でドライブに接続したら、以下のコマンドを使ってIDを変更することができます：

➢ バス ID の変更

4.1.2 の Set_Axis_Node_ID コマンド・メッセージを参照してください。

3.1.6 パラメータの保存とバックアップ

パラメータを変更した後は、必ずパラメータを保存してください。パラメータを保存した後、ドライバ

```
odrv0.save_configuration()
```

を呼び出すと、ドライバは再起動します。

パラメータのバックアップ：

```
odrivetool backup-config "d:/test.json"
```

ここで "d:/test.json "は自由に変更できるパスとファイル名です。

```
odrivetool restore-config "d:/test.json"
```

パラメータはコマンドでリストアされます：

3.1.7 つの制御モード

6010-8は、位置制御、速度制御、トルク制御、モーション制御の各モードに対応しています。

6010-8は、位置制御、速度制御、トルク制御、モーション制御の各モードに対応しています。

(軌跡制御）、円弧位置制御をサポートしています；

速度制御モードでは、速度制御とランプ速度制御をサポートしています；

トルク制御モードでは、ダイレクトトルク制御（トルク制御）とランプトルク制御（ランプトルク制御）をサポート。

モーション制御モードは、位置、速度、トルク制御モードの組み合わせであり、通常、ロボットの膝関節のような瞬間的な爆発力を必要とするシナリオで使用される。業界では、このモーション制御モードを使ってモーターを制御することから、MIT Open Source Robot Dogに由来するMIT制御モードと呼ぶユーザーもいる。

各制御モードのその後の詳細な説明では、ホストコンピュータとUSB制御命令でこのドキュメントを行う例だけでなく、同じ制御を行うために利用可能な通信プロトコル（CANなど）、ロジックは同じです。

モーターを回転させるには？

◆ モーターを始動する（クローズドループ制御に入る）

その後のすべての制御操作でモーターを回転させるには、モーターを閉ループ制御にする必要があります。
モーターウィザードで、"Start motor "をクリックしてください。odrivetoolでは以下のようになります：

```
odrv0.axis0.requested_state=8
```

◆ モータの停止（アイドル状態への移行）

モーターを停止させたい場合、またはパラメーターを変更したり保存したい場合は、モーターをアイドル状態にする必要があります。これは、モーターウィザードで "Stop motor "をクリックするか、odrivetoolで以下のコマンドを使用することで行えます：

```
odrv0.axis0.requested_state=1
```

◆ モタウザド

モーターウィザードを開くと、下部に4つのTABスイッチがあります。これは4つの制御モード、すなわち "速度制御"、"トルク制御"、"位置制御"、"MITモード "を表しています。「MITモード

➢ **インターフェイスの基本操作**

モーターを始動すると、ダイヤルがカラーで強調表示され、ほとんどすべての操作がダイヤル上で行われます：

✓ ダイヤルのクリック

ダイヤルをクリックすると、現在のダイヤル値に対する速度/トルク/位置の制御を示します。各クリックはコントロールコマンドを表します。連続して複数回クリックすることで、モーターを連続的に制御することができます。

✓ ダイヤルの右クリック

ダイヤル上で右クリックすると、ポップアップのダイアログボックスが表示され、ユーザーは正確な目標速度/トルク/位置値を入力し、Enterすると制御が実行されます。



✓ ダイヤルのホールド＆ドラッグ

ダイヤルを押したままドラッグすると、連続制御が実行され、速度/トルク/位置制御コマンドがモータに連続的に送信されます。

➢ **ドライブパラメータの変更**

Drive Parameters "をクリックすると、制御モード、速度制限、3ループゲイン（帯域幅）などのドライブパラメータを調整することができます。最初にモータを停止させてから、パラメータを変更し同期させることをお勧めします。

◆ オドリベツール

➢ **フィルター付き位置制御 (FPC)**

フィルター付き位置制御は、ユーザーが独自の位置カーブを生成し、一定の周波数で位置制御コマンドを送信したい場合に推奨されます。この場合、台形カーブ位置制御を使用すると、モーター回転が吃音や粒状になる危険性がある。

このモードでは、送信される命令の周波数に応じてフィルター帯域幅を調整する必要があります。より良いコントラストを得るには、帯域幅を

```
odrv0.axis0.controller.config.input_filter_bandwidth= 25
```

は、コマンド周波数（Hz）の半分です。例えば、コマンドが50 Hzで送信される場合：フィルタの位置制御を有効にします：

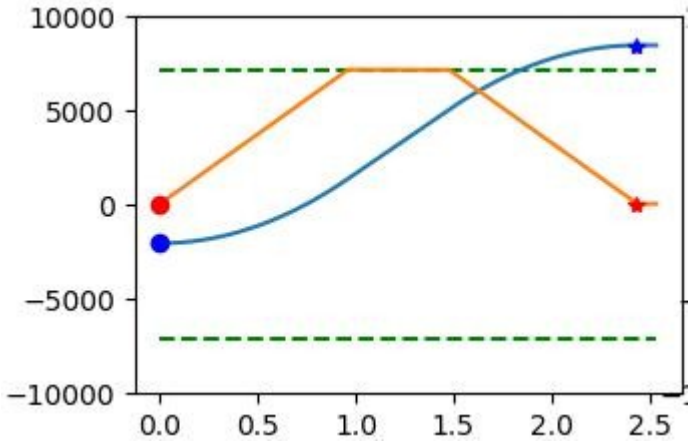
次に位置制御を有効にする：

```
odrv0.axis0.controller.config.control_mode=3
odrv0.axis0.controller.config.input_mode=3
```

```
odrv0.axis0.controller.input_pos= 10 # 単位回転数
```

- **台形カーブ位置制御（軌道制御）**

このモードでは、加速度、滑空速度、減速度を設定し、ある位置から別の位置へスムーズに移動するようにモータを制御します。台形」とは、下図のように速度カーブが台形に見えることを指し、速度がオレンジ色、位置が青色で表示されます：




```
odrv0.axis0.trap_traj.config.vel_limit # 最大滑空速度（ターン/秒） odrv0.axis0.trap_traj.config.accel_limit # 最大加速度（ターン/秒^2） odrv0.axis0.trap_traj.config.traj.config.decel_limit # 最大減速度 (turn/s^2)
odrv0.axis0.controller.config.inertia # 慣性 (Nm/(turn/s^2))
```

調整可能な制御パラメータ：

イナーシャ x 加速度 = トルクであり、デフォルトは 0 であることに注意。この値はシステム応答を改善するが、モータの負荷に直接関係する。また、前のセクションで述べた電流しきい値と速度しきい値は、依然としてグローバルに有効であることに注意してください。

また、上記の電流しきい値と速度しきい値はグローバルでも機能することに注意する必要があります。 例えば、上記の滑空速度の最大値がシステムレベルのvel_limitよりも高く設定されている場合、グローバルのvel_limitは機能します。

```
odrv0.axis0.controller.config.control_mode=3
odrv0.axis0.controller.config.input_mode=5
```

台形カーブ制御モードを有効にするには、最初

に：次に位置制御：

- **周期位置制御（円形位置制御）**

```
odrv0.axis0.controller.input_pos= 10＃単位回転数
```

このモードは連続的なインクリメンタル位置制御に適しています。例えば、ロボットのハブがある一定時間オリエンテーション接続に向かって回転したり、コンベアトラックが常時走行したりするような場合です。 通常の位置制御モードを使用すると、目標位置が徐々に非常に大きな値まで増加するため、浮動小数点精度の問題により位置決めが不正確になるという誤差が生じます。

```
odrv0.axis0.controller.config.circular_setpoints=1
```

有効にする：

このモードでは、各ミニステップは1つの円内にあり、input_posは範囲[0,1]にある。input_posがこの範囲を超えて増加すると、1周以内の値に変換される。ユーザーが1ステップを1回転以上にしたい場合は、以下のパラメータに1より大きい数値を設定することができる：

```
odrv0.axis0.controller.config.circular_setpoint_range=<N>
```

- **直接速度制御**

```
odrv0.axis0.controller.config.control_mode=2
odrv0.axis0.controller.config.input_mode=1
```

このモードは最も単純な速度制御であり、次のように有効に

します：次に目標速度を入力して制御します：

```
odrv0.axis0.controller.input_vel= 10＃unit turn/s
```

➢ **ランプ速度制御（RVC）**

ランプ速度制御モードとは、一定の勾配に従って速度を目標値まで徐々に増加させるモードで、上記の直接速度制御よりも便利です。

```
odrv0.axis0.controller.config.control_mode=2
odrv0.axis0.controller.config.input_mode=2
```

以下のように、イージングとイネーブルを追加します：

勾配を調整することで加速度を制御する：目

標速度が制御のために入力される：

```
odrv0.axis0.controller.input_vel = 10 #ターン/秒単位の勾配
```

➢ **ダイレクトトルク制御（トルク制御）**

```
odrv0.axis0.controller.config.control_mode=1
odrv0.axis0.controller.config.input_mode=1
```

これは最も単純なトルク（電流）制御モードで、次のように有効にします：

トルク制御の単位は Nm であり、ドライバファームウェアの電流の単位は A です。したがって、ドライバが Nm を電流に変換して、要求に応じて トルクを供給するようにモータを駆動できるように、トルク定数も設定する必要があります。

```
# トルク定数はおよそ8.23に等しい。
odrv0.axis0.motor.config.torque_constant= 8.23/12.3
```

```
odrv0.axis0.controller.input_torque =  1.2 # unit Nm
```

その後、制御のための目標速度が入力されます：

また、トルクモードで最高速度を制限したい場合は、enable_torque_mode_vel_limitをオンにし、
vel_limitを次のように設定することができます：

```
odrv0.axis0.controller.config.enable_torque_mode_vel_limit = 1 odrv0.axis0.controller.config.vel_limit = 30 # unit turn/s
```

➢ **ランプトルク制御**

```
odrv0.axis0.controller.config.control_mode=1
odrv0.axis0.controller.config.input_mode=6
```

ランプ・トルク制御はランプ速度制御とよく似ているので、次のように有効にしてください： 傾きを次のように調整します：

```
odrv0.axis0.controller.config.torque_ramp_rate = 0.1 # 傾斜速度の単位は Nm/s です。
```

➢ **モーションコントロール（MIT コントロール）**

モーション・コントロール・モードは、位置、速度、トルクを組み合わせて目標位置へのモータの動きを制御するモードで、以下の式で表すことができます：

$$_{arg}=\times_{i}+\times_{i}+$$

$$_{i}=_{arg}-\ _{urr}$$

$$_{i}=_{arg}-\ _{urr}$$

ここで、$_{\alpha_{\beta}}$は目標モーメント、$_{i}$は位置誤差、$_{i}$は速度誤差、 は位置制御ゲイン速度制御ゲイン（または減衰係数）、はフィードフォワードモーメントです。

```
odrv0.axis0.controller.config.control_mode=3
odrv0.axis0.controller.config.input_mode=9
```

モーション・コントロール・モードは以下のように有効になります：ゲインを調整します：

```
odrv0.axis0.controller.input_pos = 5 # 単位ターン odrv0.axis0.controller.input_mit_kp = <float> # 位置ゲイン（Nm/ターン） odrv0.axis0.controller.input_mit_kd = <float> #ダンピング係数（Nm/turn/s
```

次に、input_pos, input_vel, input_torque を入力することでモーションを制御する：

USB制御で入力される位置、速度、トルクはローター側を指すのに対し、CANを使用するMIT制御では、MITオープンソースプロトコルとの一貫性を保つため、プロトコル内の位置、速度、トルクは入力軸側を指すことに注意してください！

3.1.8 上級者向け

1 共通コマンドリスト

接続が成功すると、ユーザはコマンドによってモータを制御したり、モータの動作パラメータを取得したりすることができます。以下の表は、一般的に使用されるコマンドと、チューニングプロセスと指示を示しています：

タイプ	コマンド	命令
基本命令	dump_errors(odrv0)	すべてのエラーメッセージを表示
	odrv0.clear_errors()	すべてのエラーメッセージを消去
	odrv0.save_configuration()	この手順は、パラメータを変更した後、またはモーターがパラメータを自動認識した後、またはパラメータを較正した後に必ず実行してください。 パラメータを変更した後、またはモータがパラメータを自動認識した後、またはパラメータを校正した後、必ずこのコマンドを実行して変更内容を保存してください。
	odrv0.reboot()	ドライブを再起動する。
	odrv0.vbus_voltage	電源電圧（V）を取得
	odrv0.ibus	電源電流を取得 (A)
	odrv0.hw_version_major	ハードウェア・メジャー・バージョン番号、6010-8 現在のメジャー・バージョン番号は 3
	odrv0.hw_version_minor	ハードウェア・マイナー・バージョン番号、6010-8 現在のマイナー・バージョン番号は 8
	odrv0.hw_version_variant	同じハードウェア構成で異なる型番、6010-8は1に対応 6010-8に対応する型番は1
	odrv0.can.config.r120_gpio_num	CANインターフェースを制御する120R整合抵抗スイッチのGPIO番号。 GPIO番号
	odrv0.can.config.enable_r120	CANインターフェースを制御する120R整合抵抗スイッチのGPIO番号。
	odrv0.can.config.baud_rate	CANのボーレート設定
パラメータ設定コマンド	odrv0.config.dc_bus_undervoltage_trip_l レベル	低電圧アラームしきい値（V）
	低電圧アラームしきい値 (V) 過電圧アラームしきい値 (V)	過電圧アラームしきい値 (V)
	odrv0.config.dc_max_positive_current	最大（正）ライン電流 (A)
	odrv0.config.dc_max_negative_current	最大（負）ライン電流逆電荷 (A)
	odrv0.axis0.motor.config.resistance_cali b_max_voltage	モータパラメータ認識用の最大電圧値で、通常は電源電圧の半分よりわずかに低い値（例：24V）。 一般に、この値は電源電圧の半分よりわずかに小さい。 10に設定可能
	odrv0.axis0.motor.config.calibration_curr エント	モータパラメータを識別するときの最大電流値、この値は一般に2~5Aに設定できる。 この値は大きすぎず小さすぎず、2~5Aに設定できる。
	この値は、大きすぎず小さすぎず、2~5A に設定できます。 nt	モータのトルク定数(Nm/A)

	odrv0.axis0.min_endstop.config odrv0.axis0.max_endstop.config	最小（LW1）/最大（LW2）リミットスイッチの設定。 Enabled: 有効にするかしないか： enabled：有効にするかしないか gpio_num: 対応するIO番号、最小リミットIO番号を1、最大リミットIO番号を2に設定してください。
	odrv0.axis0.encoder.config.index_offset	この値は、エンコーダのゼロ点からのユーザーのゼロ点のオフセットです。 この値は、エンコーダのゼロ点に対するユーザーのゼロ点のオフセットです。このオフセットを設定し、設定を保存すると、すべてのユーザ入力位置制御目標値は、ユーザゼロを基準とします。
	odrv0.axis0.motor.motor_サーミスタ.con 図	モータ温度センサを設定する： enabled：有効
	odrv0.axis0.motor.fet_thermistor.config	temp_limit_lower: 下限温度 temp_limit_upper: 温度上限値
	odrv0.axis0.motor.motor_thermistor.tem 温度	モーターの温度
	モータの温度。 温度	ドライブ温度
キャリブレーションコマンド	odrv0.axis0.requested_state=4	相抵抗、相インダクタンス、および 3 相電流バランスの 🐶較正を含む、モータのパラメータを特定します。このプロセスには3～6秒かかり、モーターは 🐶鋭い音を発します。音が鳴り止んだ後、あるいは 6 秒経っても音が鳴らない場合は、dump_errors(odrv0) を実行してエラーがないかチェックする。 を実行し、エラーがないことを確認してから先に進む。
	odrv0.axis0.requested_state=7	エンコーダを較正する。この操作を実行する前に、モータ出力 🐶 軸に負荷がかかっていないこと、およびモータが手などで固定されていることを確認します。この操作を行った後、エンコーダの識別と校正を行うため、モータは一定時間正逆回転します。モータが停止した後、dump_errors(odrv0) を実行してエラーの有無を確認し、エラーがないことを確認する。 エラーがないことを確認する。
	odrv0.axis0.encoder.config.pre_calibrate d=1	pre_calibrate d=1 を書き込むと、デバイスの電源を入れるたびに較正する必要がないことを示します。このパラメータは、上記の較正が成功した後にのみ書き込むことができる。 このパラメータは、上記の較正が成功した場合にのみ書き込むことができ、そうでない場合は書き込みに失敗する。
	odrv0.axis0.controller.config.load_encod er_axis=0	現在動作しているモータがモータ 0 であることを確認する。 この操作はBETAでのみ必要である。 この操作は BETA でのみ必要であり、製品版では何の効果もない。
	odrv0.axis0.requested_state=1	モータを停止し、アイドル状態にする。
	odrv0.axis0.requested_state=8	モータ起動、クローズドループ状態
	odrv0.axis0.motor.config.current_lim	最大モータ電流（A）、この値を超えると過電流アラームが報告されます。 この値を超えると過電流アラームが発生します。**この値は**

制御コマンド		**100 を超えてはならないことに注意してください。**
	odrv0.axis0.controller.config.vel_limit	モータの最高回転速度（turn/s）。モータのロータ速度がこの値を超えると、速度超過アラームが発生します。 モータロータ速度がこの値を超えると、速度超過アラームが発生する。
	odrv0.axis0.controller.config.enable_vel_limit リミット	速度制限スイッチ。Trueの場合、上記のvel_limitは無効になる。 上記のvel_limitはTrueの時に有効になり、Falseの時は無効になる。
	odrv0.axis0.controller.config.control_mo de 制御モード。	制御モード。 0: 電圧制御 1: トルク制御 2: 速度制御 3: 位置制御
	入力モード。	入力モード。モータ動作を制御するために、ユーザが制御値を入力するモードを示す。 0: アイドル 1: 直接制御 2: 速度ランプ 3：位置フィルタ 5：台形カーブ
		6：トルクランプ 9: モーションコントロール(MIT)
	odrv0.axis0.controller.config.vel_gain	速度ループPID制御のP値
	odrv0.axis0.controller.config.vel_integrat or_gain	速度ループPID制御のI値
	odrv0.axis0.controller.input_mit_kp	モーションコントロール（MIT）の位置ゲイン
	odrv0.axis0.controller.input_mit_kd	モーションコントロール（MIT）速度ゲイン（ダンピング係数）
	odrv0.axis0.controller.config.pos_gain	位置ループPID制御のP値
	odrv0.axis0.controller.input_torque	トルク制御、または速度制御/位置制御の目標値 トルクフィードフォワード（Nm）
	odrv0.axis0.controller.input_vel	速度制御のターゲット、または位置制御の速度フィードフォワード (ターン/秒)
	odrv0.axis0.controller.input_pos	位置制御のターゲット（ターン）
	odrv0.axis0.encoder.set_linear_count()	エンコーダの絶対位置を設定する。括弧内に32ビット整数を入力し、この整数の絶対値は エンコーダの絶対位置を設定します。

	odrv0.axis0.trap_traj.config	3つのパラメータを含む： ➢ accel_limit:最大加速度(rev/s^2) ➢ decel_limit：最大減速度 (rev/s^2) ➢ vel_limit：最大速度（rev/s） これら3つのパラメータは、odrv0.axis0.controller.config.input_modeが台形カーブの場合に機能します。 台形カーブの場合、位置制御を調整する。 位置制御の加減速効果を調整する。
	odrv0.axis0.controller.config.input_filter_bandwidth	input_filter_bandwidthパラメータは、odrv0.axis0.controller.config.input_modeが位置フィルタの場合に機能し、位置制御の加減速効果を調整します。 位置制御の加減速効果を調整します。

2 グラフィカルチューニング

モータのチューニング時に特定の動作パラメータをリアルタイムで監視する必要がある場合、pythonの強力な計算およびグラフィカルライブラリとType-Cインターフェースの高速スループット機能を活用して、🐶モータパラメータをリアルタイムで出力することができます。

1. 環境準備

```
pip install numpy matplotlib
```

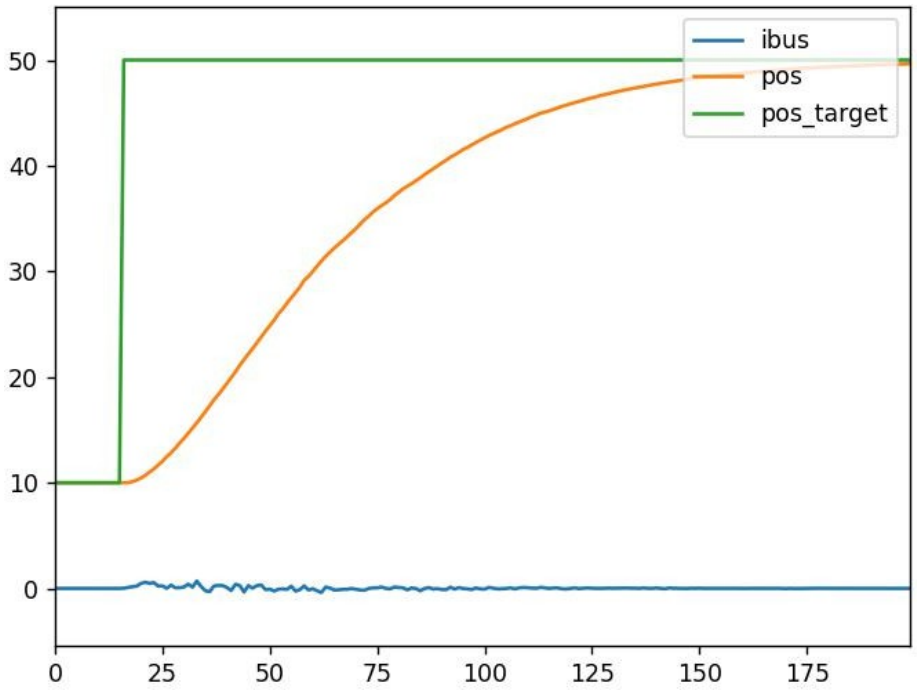
計算・グラフィカルライブラリをインストールします：

2. グラフィカルなパラメータ入力🐶

```
start_liveplotter(ラムダ:[odrv0.ibus,odrv0.axis0.encoder.pos_estimate, odrv0.axis0.controller.input_pos],["ibus",
"pos", "pos_target"])
```

odrivetoolコマンドライン・インターフェイスで、グラフィック・ライブラリを表示し、以下のようなモーター動作のメトリクスを読み取ります：

このコマンドを実行すると、グラフィカル・インターフェースが表示され、次の3つのインジケータがリアルタイムで失われます✚：ライン電流、位置、目標位置。次に、モーターの位置制御が実行され、モーターのリアルタイムの位置制御曲線が表示されます：



3 USBとCANの互換性

本製品の以前のバージョン(ハードウェア・バージョンが3.7以下。次のサブセクション3.1.8の odrv0.hw_version_majorおよびodrv0.hw_version_minorコマンドで取得できます)では、USBはCANと互換性がありません。USBとCANは互換性がありません:)

➢ USB通信中のCANへの切り替え

CANが無効になっている場合、Type-Cインターフェイスを使用して通信を行うことができる：

```
odrv0.config.enable_can_a= True odrv0.axis0.requested_state=
AXIS_STATE_IDLE odrv0.save_configuration()
```

➢ CAN通信中のUSBへの切り替え

CANが有効になっている場合、ユーザーはまずSet_Axis_Stateメッセージ(アイドル(IDLE)状態のパラメータ1)を送信してモーターをアイドル状態に切り替え、次にDisable_Canメッセージ(4.1.2参照)を送信してUSBに切り替えます。

USBからCAN、CANからUSBのいずれに切り替える場合でも、モーターをアイドル(IDLE)状態にする必要があります。

4 CANマッチング抵抗スイッチ

120Ωのインピーダンス整合抵抗はすでにドライブに搭載されており、必要に応じてユーザーがオン/オフを切り替えることができます。

```
odrv0.can.config.r120_gpio_num= 5
odrv0.can.config.enable_r120= True
```

を以下のように設定する：

5 ユーザーゼロ設定

デフォルトでは、モータから読み出されるユーザ位置と、位置制御を行うときの入力値は、ドライ ブ上のアブソリュート・エンコーダのゼロ点を基準としています。しかし、ユーザシナリオでは、エンコーダのゼロ点はユーザのゼロ点ではないことがほとんどであるため、ユーザはこのゼロ点オフセットを手動で設定する必要があります。

一般的に、ユーザーがこのゼロ点を見つけることができる2つの手段があります。1つはリミットスイッチを使用する方法、もう1つはゼロオフセット（エンコーダのゼロ点に対するユーザーのゼロ点のオフセット値）を手動で設定する方法です：

```
# 手動または位置制御によって、ユーザーが最初に希望のユーザーゼロ位置に回転した後：
odrv0.axis0.encoder.config.index_offset=  odrv0.axis0.encoder.pos_estimate
```

6 リミットスイッチ

ドライバは2つのリミットスイッチ（LW1とLW2）をサポートしています。

```
odrv0.axis0.min_endstop.config.enabled= True odrv0.axis0.min_endstop.config.gpio_num= 1
odrv0.axis0.max_endstop.config.enabled= True odrv0.axis0.max_endstop.config.gpio_num= 2
```

最大位置。つのリミットスイッチを使用するには、以下の構成を使用します：

リミットスイッチがトリガされると、システムはMIN_ENDSTOP_PRESSED または MAX_ENDSTOP_PRESSED エラーを報告し、ホストコンピュータはこの時点で関連する操作を実行できます。

リミットスイッチ機能はハードウェアバージョン3.7ではサポートされていないことに注意してください。

7 ブレーキ抵抗

ユーザーは、2.4.7 で説明されているように、ブレーキ抵抗インターフェース（EMG と GND）の間にブレーキ抵抗（ブリーダー抵抗とも呼ばれる）を接続し、バス電流が電源にバックアップされるときの瞬時消費電 流をシャントすることができる。

```
odrv0.config.enable_brake_resistor = True odrv0.config.brake_resistance = xxx # 外部ブレーキ抵抗の抵抗値を設定 odrv0.save_configuration()
```

ユーザは以下のコンフィギュレーションで有効にすることができる：

ドライバを自動的にドレインさせるには？以下の2つの方法を組み合わせることで、コントローラが自動的にドレイン電流を調整できるようになります：

```
odrv0.config.max_regen_current = xxx # 最大逆電流、正の数
```

➢ 最大逆電流を調整することにより

逆電流が上記設定値より大きい場合、ドライバは過剰電流をドレイン抵抗にシャントします。

```
odrv0.config.enable_dc_bus_overvoltage_ramp = True odrv0.config.dc_bus_overvoltage_ramp_start = xxx # シャントする電圧の下限 odrv0.config.dc_bus_overvoltage_ramp_end = xxx # シャントの上限電圧
```

➢ 逆電位電圧範囲の調整により

逆起電力による電圧が下限電圧以上になるとドレイン抵抗にシャントを開始し、上記上限電圧以上になるとドレイン抵抗に全電圧を印加してシャントを試みます。下限電圧と上限電圧の間にある場合は、上記のようにドレイン抵抗に電源電圧（逆起電力電圧）を印加します。

なお、上記2つのモードは同時に有効にすることができ、ドライバは2つの方法を組み合わせて放電を試みます。

3.2 ファームウェアアップデートダウンロード

ファームウェアは、SWDコネクタ（2.4.4）またはType-Cコネクタ（2.4.2）を介して、以下の3つの方法で書き込むことができます：

3.2.1 ナショナルダウンロードソフトウェア

1. USB（DFU）書き込み

Nationwideダウンロード・ソフトウェアは、Type-Cインターフェイスまたは SWDインターフェイス（JLinkとDAPのみサポート）を介して書き込みが可能であり、本セクションでは主にType-Cインターフェイスの書き込みを例として説明します。

まず、Nationwide書き込みソフト（https://www.cyberbeast.cn/filedownload/789489）のUSBドライバをダウンロードし、対応するシステムのドライバをインストールします。次に、Nationwide書き込みソフト（https://cyberbeast.cn/filedownload/766844）をダウンロードし、任意のディレクトリに解凍して実行します。次に、Nationwide書き込みソフトウェア（）をダウンロードし、任意のディレクトリに解凍して実行する。

次に、Type-Cコネクタを接続し、odrivetoolに入り、以下のコマンドを実行してドライブをDFUモードにする：

```
odrv0.enter_dfu_mode()
```

最後に、ナショナル・バーナー・ソフトウェアを使用して、下図のようにドライブを書き込みます。書き込みが終わったら、"Common Operation "をクリックし、"Reset "をクリックしてドライブを再起動し、テストのためにodrivetoolに接続してください。



2. SWD(JLinkまたはDAP)書き込み

ダウンロードにSWDを使用するのはDFUモードと似ていますが、SWDデバッグインターフェイス(2.4.4)を通して接続し、上の図で対応するデバッグツール(JLinkまたはDAP)を選択する必要があります。

3.2.2 pyocd

pyocdはopenOCDのpythonバージョンで、STLink、JLink、DAP、その他の一般的なデバッグツールをサポートし、消去、書き込み、リセット、その他の操作を行うことができます。**ドライブは SWD インターフェースで接続する必要があります。**SWDの配線順序は2.4.4を参照。 **SWDコネクタには3.3Vの電源があるので、ドライブの損傷を避けるために間違った順序で配線を接続しないでください！**

```
pip install pyocd
```

1. pyocd をインストールする
2. 焼く

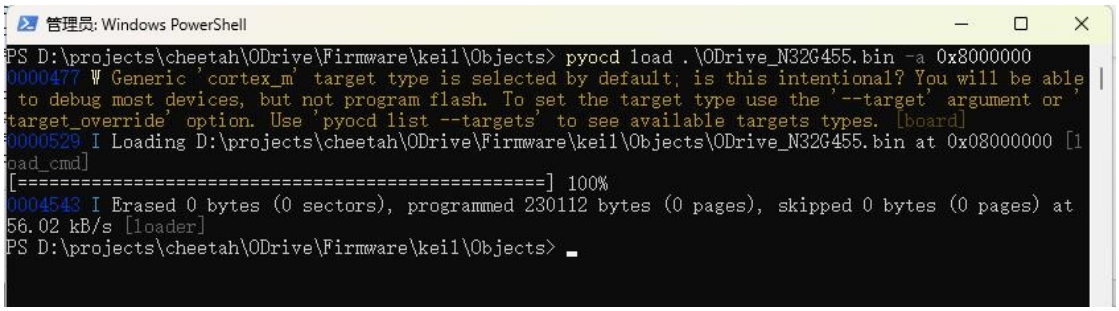
```
pyocd リスト
```

まず、🔗接続しているデバッグツールをリストアップします：



次に、以下のコマンドを実行して bin ファイルを書き込みます：

```
pyocd load .\pyocd load .
```



4 通信プロトコルと例

4.1 CAN シンプルプロトコル

デフォルトの通信インターフェースはCANで、最大通信速度は1Mbps（odrv0.can.config.baud_rateで読み込んで設定可能）、デフォルトの通信速度は500Kbpsです。 **注意：USBは、以前のハードウェアバージョン（3.7以下）のCANとは互換性がありません。3.1.8の3)項を参照してください。USBからCANへの切り替え方法。**

ドライブのデフォルトプロトコルはCAN Simpleであり、ユーザーがCANOpenプロトコルに切り替える場合は、以下の手順で行う必要があります。

```
odrv0.can.config.protocol = 1
```

コマンドを実行し、CAN Simpleに切り替える必要があります：

通信インターフェイスは、3.10以上のハードウェアバージョンでRS485、パルス方向、およびPWMの両方をサポートしていることに注意してください。以下のコマンドを使用して、通信インターフェイスがCANであることを確認してください：

モーターウィザードでは、通信パラメータで物理インターフェースとCANプロトコルバージョンを切り替えることもできます。

```
odrv0.config.comm_intf_mux= 0
```

4.1.1 プロトコルフレームフォーマット

CAN通信では、下表に示す標準フレームフォーマット、データフレーム、11ビットID、8バイトのデータを使用します（左がMSB、右がLSB）：

データフィールド	CAN ID（11ビット）		データ（8バイト）
セグメンテーション	ビット10~ビット5	Bit4~ Bit0	バイト0~ バイト7
説明	ノードID	cmd_id	通信データ

- node_id: このモータのバス上のユニークなIDを表し、odrv0.axis0.config.can.node_idを使ってodrivetoolで読み取り、設定することができます。
- cmd_id: コマンドコード、プロトコルのメッセージタイプを示す。
- 通信データ：8バイト、メッセージに含まれる各パラメーターは、整数または浮動小数点数としてエンコードされます。バイトオーダーはリトルエンディアンで、浮動小数点数はIEEE 754標準に従ってエンコードされます（https://www.h-schmidt.net/FloatConverter/IEEE754。浮動小数点数はIEEE 754標準に従ってhtml、エンコードをテストできます）エンコードされます（.でエンコードのテストができます）。

4.1.2で説明したSet_Input_Posメッセージを例にして、その3つのパラメータがInput_Pos=3.14、Vel_FF=1000（1rev/sの意 味）、Torque_FF=5000（5Nmの意味）であり、Set_Input_PosメッセージのCMD IDが0x00Cであり、ドライブのノード（node_FF）が0x00Cであるとします。ドライバのnode_idが0x05に設定されているとする：

- 11ビットCAN ID = (0x05<<5)+0x0C=0xAC
- 4.1.2のSet_Input_Posの記述によると、Input_Posは0バイト目から4バイトがC3 F5 48 40(浮動小数点3.14はIEEE754規格で32ビット数0x4048f5c3としてエンコードされる)、Vel_FFは4バイト目から2バイトがE8 03(1000=0x03E8)としてエンコードされる。1000=0x03E8）、Torque_FFは6バイト目先頭の2バイトを88 13（5000=0x1388）とエンコードし、8バイトの通信データとなる：

バイト0	バイト0	バイト2	バイト3	バイト4	バイト5	バイト6	バイト7
バイト7	バイト4 バイト5 バイト6 バイト7 バイト	48バイト	40	E8	03	88	13

4.1.2 フレームメッセージ

次の表は、使用可能なすべてのメッセージの一覧です：

CMD ID	名前	方向	パラメータ
0x001	ハートビート	モーター→ ホスト	軸エラー 軸状態 モータフラグ エンコーダフラグ コントローラフラグ Traj_Done モータフラグ エンコーダフラグ コントローラフラグ
0x002	ライフ 0x002	ホスト→ モータ	
0x003	Get_Error	モーター→ ホスト	エラータイプ
0x004	RxSdo	モーター→ ホスト	
0x005	TxSdo	モーター→ メインフレーム	
0x006	軸ノード ID の設定	ホスト→ モータ	軸ノード ID
0x007	軸の状態設定	ホスト→ モータ	軸要求状態
0x008	制御	ホスト→ モータ	

0x009	Get_Encoder_Estimates	モータ→ ホスト	Pos_Estimate Vel_Estimate
0x00A	Get_Encoder_Count	モーター→ ホスト	シャドウカウント カウント
0x00B	コントローラモードの設定	ホスト→ モーター	制御モード 制御モード
0x00C	Set_Input_Pos	本体→ モータ	入力位置 Vel_FF トルク
0x00D	Set_Input_Vel	ホスト→ モータ	入力 トルク
0x00E	入力トルク設定	本体→ モータ	入力トルク
0x00F	リミット設定	ホスト→ モータ	速度リミット 電流リミット
0x010	アンチコギング開始	ホスト→ モータ	
0x011 Set_Traj_Vel_Limit	設定_Traj_Vel_Limit	ホスト→ モーター	Set_Traj_Vel_Limit
0x012 Set_Traj_Accel_Limits	アクセル・リミットの設定	ホスト→ モーター	設定_Traj_Accel_Limits Traj_Decel_Limit（トラジ・ディセル・リミット
0x013	セット_トラジ_イナーシャ	メインフレーム→ モーター	慣性
0x014	Get_Iq	モーター→ ホスト	Iq_Setpoint Iq_測定値
0x016	再起動	ホスト→ モーター	
0x017	Get_Bus_Voltage_Current	モーター→ ホスト	バス電圧 バス電圧
0x018	エラーをクリア	ホスト→ モーター	
0x019	Set_Linear_Count	ホスト→ モーター	リニアカウント
0x01A	Set_Pos_Gain	本体→ モーター	ポジションゲイン
0x01B	セット_Vel_ゲイン	ホスト→ モーター	ゲイン Vel_Integrator_Gain
0x01C	Get_Torques	モーター→ ホスト	トルク設定値 トルク
0x01D	Get_Powers	モーター→ ホスト	電気的パワー メカニカルパワー
0x01E	無効化	ホスト→ モータ	
0x01F	設定の保存	ホスト→ モーター	

すべてのメッセージの詳細な説明を以下に示します：

➢ ハートビート

CMD ID: 0x001 (モーター→ ホスト)

0.5.11未満（および0.5.11を含む）のファームウェアバージョンには、以下のフォーマットがあります：

開始バイト	開始バイト	タイプ	説明
0	軸エラー	uint32	ドライバ例外コード（odorv0.axis0.error）
4	軸の状態	uint8	ドライバの状態（odrv0.axis0.current_state）
5	モーターフラグ	uint8	1：モーター異常（odorv0.axis0.motor.errorが0ではない） 0：モーター正常（odrv0.axis0.motor.errorが0）
6	エンコーダフラグ	uint8	1：エンコーダ異常（odrv0.axis0.encoder.errorが0ではない） 0: エンコーダ正常(odrv0.ax0.encoder.error is 0)
0：エンコーダ正常（odrv0.ax0.encoder.errorが0）	コントローラフラグ	uint8	bit7odrv0.axis0.controller.trajectory_done,すなわち、位置プロファイルが実行されたかどうか。 bit0： 1：制御異常（odrv0.axis0.controller.error not 0） 0：制御正常（odrv0.axis0.controller.error が 0）

ファームウェアのバージョンが0.5.12以上（0.5.12を含む）の場合の心跳のフォーマットは以下の通りです：

開始バイト	開始バイト	タイプ	説明
0	軸エラー	uint32	ドライバ例外コード（odorv0.axis0.error）
4	軸の状態	uint8	ドライバの状態（odrv0.axis0.current_state）
5	フラグ	uint8	bit0：モータ例外ビット（odrv0.axis0.motor.errorが0かどうか） bit1：エンコーダ例外ビット（odrv0.axis0.encoder.errorが0かどうか） bit2：制御例外ビット（odrv0.axis0.controller.errorが0かどうか） bit7：odrv0.axis0.controller.trajectory_done、つまりビット bit7：odrv0.axis0.controller.trajectory_done。
6	予約済み	uint8	予約済み
7予約	ライフ	uint8	定期メッセージのライフ値にホップごとに1を加えた値で、範囲は0〜255。 0〜255、ライフ値が連続していない場合、心跳消息が失われた、つまり通信が不安定であることを意味する。 life値が連続していない場合、ホップが失われた、すなわち通信が不安定であることを意味する。

ファームウェアのバージョンが0.5.13以上の場合（0.5.13を含む）、フォーマットは以下のようになる：

スタートバイト	名前	タイプ	説明
0	軸エラー	uint32	ドライバ例外コード（odorv0.axis0.error）
4	軸の状態	uint8	ドライバの状態（odrv0.axis0.current_state）

5	フラグ	uint8	bit0：モータ例外ビット（odrv0.axis0.motor.errorが0かどうか） bit1：エンコーダ例外ビット（odrv0.axis0.encoder.errorが0かどうか） bit2：制御例外ビット（odrv0.axis0.controller.errorが0かどうか） bit3：システム例外ビット（odrv0.errorが0かどうか）
			bit7: odrv0.axis0.controller.trajectory_done, bit 設定曲線の実行が終了したかどうか
6	予約済み	uint8	予約済み
7予約	ライフ	予約	0～255の範囲で、各ホップごとに1ずつ加算される定期メッセージの寿命値。 0～255、このライフ値が連続していない場合、サービスホップメッセージが失われた、つまり通信が不安定であることを意味する。 この値が連続していない場合は、ホップが失われた、つまり通信が不安定であることを意味する。

➢ エストップ

CMD ID: 0x002 (ホスト→ モータ) パラメータなし、データなし。

このコマンドは緊急モータ・シャットダウンを引き起こし、ESTOP_REQUESTED 例外を報告する。

➢ Get_Error

CMD ID: 0x003 (モータ→ ホスト)

入力 (ホスト→ モータ)：

開始バイト	名前	タイプ	タイプ
0	エラータイプ	uint8	0：モーター例外を取得 1：エンコーダ例外取得 3：制御例外取得 4：システム例外を取得

➢ Lose 🐶（モーター→ ホスト）：

スタートバイト	名前	タイプ	タイプ
0	エラー	uint32/uint64	異なる入力Error_Typeは異なるデータとサイズを返す： 0：モータ例外（odrv0.axis0.motor.error）、uint64、8バイト 1：エンコーダ例外（odrv0.axis0.encoder.error）、uint32、4バイト 3：制御例外（odrv0.axis0.controller.error）、uint32、4バイト 4：システム例外（odrv0.error）、uint32、4バイト

➢ RxSdo

CMD ID: 0x004 (ホスト→ モーター)

入力

スタートバイト	名前	タイプ	タイプ
0	オペコード	uint8	0:読み取り 1：書き込み
1	エンドポイントID	uint16	すべてのパラメータとインターフェース関数のIDが記載されたJSONファイルをダウンロードしてください： ➢ バージョン 0.5.13 https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.js ➢ バージョン0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.js オン
3	予約済み	uint8	
4	値	uint8[4]	上記のJSONに記述されているように、Endpoint_IDによって異なる。 Endpoint_ID が読み書き可能な float 値に対応する場合、ここでの 4 バイトが IEEE エンコードされた float 値となる。 opcode=1 の場合、この float 値に値が書き込まれる。

出力🐶（上記 opcode=0 の場合）：

開始バイト	名前	タイプ	出力
0	オペコード	uint8	0に固定
1	エンドポイントID	uint16	すべてのパラメータとインターフェース関数のIDが記載されたJSONファイルをダウンロードしてください： ➢ バージョン 0.5.13 https://bl.cyberbeast.cn/actuator/endpoints_0.5.13.js ➢ バージョン0.5.14 https://bl.cyberbeast.cn/actuator/endpoints_0.5.14.js オン
3	予約済み	uint8	
4	値	uint8[4]	上記のJSONに記述されているように、Endpoint_IDによって異なる。 Endpoint_ID が読み取り可能な uint32 値に対応する場合。 の場合、ここでの4バイトはリトルエンディアンバイトオーダーのuint32である。

➢ TxSdo

CMD ID: 0x005 (モーター→ ホスト)

使用法は opcode=1 のときの RxSdo と同じ。

➢ 軸ノード ID の設定

CMD ID: 0x006 (ホスト→ モータ)

開始バイト	名前：0x006（ホスト・モータ）	名前	説明
0	軸ノードID	uint32	odrv0.axis0.config.can.node_id CANバス上のモーターの固有ID

➢ 軸の状態設定

CMD ID: 0x007 (ホスト→ モーター)

開始バイト	名前：0x007（ホストモーター）	名前	説明
0	軸要求状態	uint32	odrv0.axis0.requested_state ➢ 0: 未定義 ➢ 1: アイドル ➢ 3：較正（モーター較正＋エンコーダー較正） ➢ 4：モーター較正 ➢ 7：エンコーダ較正 ➢ 8: 閉ループ

➢ Mit_Control CMD
ID: 0x008

これはアナログMITオープンソースモーションコントロールプロトコル（https://github.com/mit-biomimetics/Cheetah-Software）の実装です。

USB制御で入力される位置、速度、トルクはローター側を指すのに対し、CANを使用するMIT制御では、MITオープンソースプロトコルとの一貫性を保つため、プロトコル内の位置、速度、トルクは出力🐶シャフト側を指すことに注意してください！

✓ ホスト→ モーター

CAN データ データフレーム ビット	意味	意味 説明
BYTE0 BYTE1	**位置**：合計16ビット、BYTE 0は上位8ビット、BYTE 1は下位8ビット。 BYTE0は上位8ビット、BYTE1は下位8ビット 出力🐶軸の多回転位置（ラジアン単位）（RAD）	実際の位置はdouble型であり、16ビットのint型に変換する必要がある： pos_int= (pos_double+ 12.5)*65535 / (pos_double 12.5)*65535 / (pos_double 12.5) 25
BYTE2 BYTE3 BYTE4	**速度**：合計12ビット、BYTE2はその上位8ビット。 BYTE3[7-4] (高 4 ビット)は下位 4 ビット。出力🐶軸の角速度を RAD/s で示す。 **KP値**：合計12ビット、BYTE3[3-0]（下位4ビット）は上位4ビット。 BYTE3[3-0](下位 4 ビット)が上位 4 ビット、BYTE4 が下位 8 ビット。	実際の**速度は**doubleであり、12ビットのintに変換する必要がある： vel_int = (vel_double + 65) * 4095 / 130 **KP値は**実際にはdoubleであり、12ビットintに変換する必要がある： KP_INT= KP_DOUBLE * 4095 / 500
BYTE5 BYTE6 BYTE7	**KD値**：合計12ビット、BYTE5が上位8ビット、BYTE6[7-4]（上位4ビット）が下位4ビット。 **トルク値**：合計 12 ビット、BYTE6[3-0](下位 4 ビット)は上位4ビット、BYTE7は下位8ビット。単位はN.m.である。	**KD値は**実際にはdoubleであり、12ビットのintに変換する必要がある： KD_INT= KD_DOUBLE * 4095 / 5 実際の**トルクは**doubleであり、12ビットのint型に変換する必要がある： t_int= (t_double+ 50)* 4095 / 100 トルク定数の単位はN.m/Aです。

✓ モータ→ ホスト

CAN データ データフレーム ビット	意味	意味
バイト0	ノードID	ドライブ・ノードID

BYTE1	**位置**：合計16ビット、BYTE1は上位8ビット、BYTE2は下位8ビット。 BYTE1は上位8ビット、BYTE2は下位8ビット	実際の位置はdouble型であり、16ビットint型から変換する必要がある： pos_double= pos_int * 25 / 65535 - - 12.5
BYTE2	出力🔗軸のラジアン単位の多回転位置 (RAD)	12.5
BYTE3	**速度**：合計12ビット、BYTE3は上位8ビット。 BYTE4[7-4] (高 4 ビット)は下位 4 ビット。出力🔗軸の角速度を RAD/s で示す。 **トルク**：合計 12 ビット、BYTE4[3-0]（下位 4 ビット）が上位 4 ビット、BYTE5 が下位 8 ビット。 は上位4ビット、BYTE5は下位8ビット。単位はN.m.。	実際の**速度は**doubleであり、12ビットのintから変換する必要がある： vel_double = vel_int * 130 / 4095 - 65 実際のトルクは double であり、以下のように 12 ビット int から変換する必要がある： t_double= t_int * 100 / 4095 - 50 トルク定数（N.m/A
BYTE4		
BYTE5		

➢ Get_Encoder_Estimates CMD ID:
0x009 (motor→ host)

開始バイト	名前	タイプ	単位	説明
0	位置推定	float32	odrv0.axis0.encoder.pos_estimate	odrv0.axis0.encoder.pos_estimate 現在のモーターローター位置
4	Vel_Estimate（推定値	float32	rev/s	0軸0.エンコーダ.vel_estimate 現在のモーターローター速度

➢ Get_Encoder_Count

CMD ID: 0x00A (モーター→ ホスト)

開始バイト	名前	タイプ	説明
シャドウカウント	シャドウカウント	int32	ODRV0.AXIS0.エンコーダシャドウカウント エンコーダの多回転カウント
4	カウント_In_Cpr	int32	ODRV0.AXIS0.エンコーダーカウントイン_Cpr エンコーダ1回転カウント

➢ Set_Controller_Mode CMD ID:
0x00B (ホスト→ モーター)

開始バイト	名前	名前	説明
0	制御モード	uint32	odrv0.axis0.controller.config.control_mode 制御モード： 0：電圧制御 1：トルク制御 2：速度制御 3：位置制御
4	入力モード	uint32	odrv0.axis0.controller.config.input_mode Input_Mode、ユーザーによる制御値の入力方法を示す

			0: アイドル 1: 直接制御 2：スピードランプ 3: ポジションフィルター 5：台形カーブ 6: トルクランプ 9: モーションコントロール(MIT)

➢ Set_Input_Pos

CMD ID: 0x00C (ホスト→ モータ)

開始バイト	名前	タイプ	単位	説明
0	入力位置	float32	odrv0.axis0.controller.input_pos。	odrv0.axis0.controller.input_pos 位置制御の目標値。
4	Vel_FF	int16	0.001rev/s	0.軸0.コントローラ.input_vel 位置制御のための速度フィードフォワード。
6	トルク_FF	int16	0.001Nm	位置制御用のトルクフィードフォワード。 位置制御のためのトルクフィードフォワード。

➢ Set_Input_Vel

CMD ID: 0x00D (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	入力値	float32	rev/s	odrv0.axis0.controller.input_vel 速度制御の目標値。
4	トルクFF	float32	Nm	odrv0.axis0.controller.input_torque 速度制御用のトルクフィードフォワード。

➢ 入力トルク設定

CMD ID: 0x00E (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	入力トルク	float32	Nm	odrv0.axis0.controller.input_torque トルク制御の目標値。

➢ リミット設定

CMD ID: 0x00F (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	速度リミット	float32	rev/s	odrv0.axis0.controller.config.vel_limit グローバルな速度制限。
4	カレントリミット	float32	A	odrv0.axis0.motor.config.current_limit

				モーターの電流制限。

➢ アンチコギング開始

CMD ID: 0x010 (ホスト→ モータ)

トルク リップル較正を実行します。

➢ Set_Traj_Vel_Limit

CMD ID: 0x011 (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	Traj_Vel_Limit（トラジ・ベル・リミット	float32	rev/s	トラップの速度制限。 台形位置制御の速度制限。

➢ Set_Traj_Accel_Limits CMD ID：
0x012 （ホスト→）。

開始バイト	名前	タイプ	単位	説明
0	アクセルリミット	float32	rev/s^2	加速度リミット。 台形位置制御の加速度制限。
4	トラップディセルリミット	float32	rev/s^2	トラペゾイド位置制御の減速度制限。 台形位置制御の減速リミット。

➢ イナーシャ設定

CMD ID: 0x013 (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	トラジ慣性	float32	Nm/(rev/s^2)	odrv0.axis0.controller.config.inertia イナーシャ。

➢ Get_Iq

CMD ID: 0x014 (モーター→ ホスト)

開始バイト	名前	タイプ	単位	説明
0	Iq_設定値	float32	A	odrv0.axis0.motor.current_control.Idq_setp oint Q軸電流目標値
4	Iq_測定値	float32	A	ODRV0.AXIS0.MOTOR.CURRENT_CONTROL.IQ_MEASURED。 Iq_measured Q軸電流の実測値。

➢ 再起動

CMD ID: 0x016 (ホスト→ モーター)

➢ Get_Bus_Voltage_Current CMD ID：
0x017 (motor→ ホスト)

開始バイト	名前	タイプ	単位	説明
0	バス電圧	float32	V	odrv0.vbus_voltage バス電圧。
4	バス電流	float32	A	odrv0.ibus バス電流。

➢ クリア_エラー

CMD ID: 0x018 (ホスト→ モーター)

すべてのエラーと例外をクリアします。

➢ Set_Linear_Count

CMD ID: 0x019 (ホス ト→ モーター)

エンコーダの絶対位置を設定します。

開始バイト	名前	タイプ	タイプ
0	リニアカウント	int32	odrv0.axis0.encoder.set_linear_count() エンコーダの絶対位置（カウント値）を設定します。

➢ Set_Pos_Gain

CMD ID: 0x01A (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	位置ゲイン	float32	(rev/s)/rev	odrv0.axis0.controller.config.pos_gain 位置リングゲインKp

➢ Set_Vel_Gains

CMD ID: 0x01B (ホスト→ モーター)

開始バイト	名前	タイプ	単位	説明
0	ゲイン	float32	Nm/(rev/s)	odrv0.axis0.controller.config.vel_gain 速度ループゲイン Kp.
4	Vel_Integrato r_Gain	float32	Nm/rev	軸0.config.vel_integrator_ゲイン 速度ループゲイン Ki。

➢ Get_Torques

CMD ID：0x01C （モーター→ ホスト）

開始バイト	名前	タイプ	単位	説明
0	トルク設定値	float32	Nm	odrv0.axis0.controller.torque_setpoint 現在のループトルク目標値。
4	トルク	float32	Nm	現在のトルク値を示す。

➢ Get_Powers

CMD ID: 0x01D (モーター→ ホスト)

開始バイト	名前	タイプ	説明
0	電力	float32	odrv0.axis0.controller.electrical_power。 電力
4	機械的電力	float32	ODRV0.AXIS0.CONTROLLER.MECHANICAL_POWER（メカニカルパワー メカニカルパワー

➢ 無効化

CMD ID: 0x01E (ホスト→ モータ)

CANを無効にしてドライブを再起動します。

➢ コンフィギュレーションの保存

CMD ID: 0x01F (ホスト→ モーター)

現在のコンフィグレーションを保存して有効にし、再起動します。

4.1.3 CANプロトコルの実践

4.1.3.1 実践：パワーアップキャリブレーション

CANメッセージを送信するシーケンスは次のとおりです：

CAN ID	フレームタイプ	フレームデータ	内容
0x007	データフレーム	04 00 00 00 00 00 00	メッセージ：Set_Axis_State パラメータ：4 モータの較正
0x007	データフレーム	07 00 00 00 00 00 00	メッセージ：Set_Axis_State パラメータ：7 エンコーダの較正

4.1.3.2 実用：速度制御

CANメッセージの送信シーケンスは次のとおりです：

CAN ID	フレームタイプ	フレームデータ	内容
0x00B	データフレーム	02 00 00 00 02 00 00 00	メッセージ：Set_Controller_Mode パラメータ：2/2 制御モードを速度制御に設定します。

			速度ランプ用
0x007	データフレーム	08 00 00 00 00 00 00 00	メッセージ： Set_Axis_State パラメータ： 8 閉ループ制御状態に入る
0x0D	データフレーム	00 00 20 41 00 00 00 00	メッセージ：Set_Input_Vel パラメータ：10/0 目標速度とモーメントフィードフォワードを設定します。目標速度は 10（浮動小数点数：0x41200000）、モーメントフィードフォワードは 0（浮動小数点数：0x00000000）です。 モーメントフィードフォワードは 0（浮動小数点数： 0x00000000）です。

4.1.3.3 実践：位置制御

CANメッセージの送信シーケンスは次のとおりです：

CAN ID	フレームタイプ	フレームデータ	内容
0x00B	データフレーム	03 00 00 00 03 00 00 00	メッセージ: Set_Controller_Mode パラメータ: 3/3 制御モードを位置制御に設定します。 は位置フィルタ
0x007	データフレーム	08 00 00 00 00 00 00 00	メッセージ： Set_Axis_State パラメータ： 8 閉ループ制御状態に入る
0x0C	データフレーム	cd cc 0c 40 00 00 00 00	メッセージ： Set_Input_Pos パラメータ： 2.2/0/0 目標位置、速度フィードフォワード、トルクフィードフォワードを設定する。目標位置は 2.2（浮動小数点数：0x400CCCD）、トルクフィードフォワード、速度フィードフォワードは 0。 フィードフォワードと速度フィードフォワードは 0 。

4.1.4 CANOpenとの互換性

ノードIDが適切に割り当てられていれば、CANOpenとの相互運用が可能です。次の表に、CANopenとこのプロトコル（🐶）で有効なノードIDの組み合わせを示します：

CANOpenノードID	プロトコルノードID
32 ... 127	0x10、0x18、0x20、0x28
64 ... 127	0x10, 0x11, 0x18, 0x19, 0x20, 0x21, 0x28, 0x29
96 ... 127	0x10, 0x11, 0x12, 0x18, 0x19, 0x1A, 0x20, 0x21, 0x22, 0x28, 0x29, 0x2A

4.1.5 サイクルメッセージ

ユーザは、ホストコンピュータがモータに要求メッセージを送信することなく、モータがホストコンピュータに周期的メッセージを送信するように設定できます。周期メッセージは、以下の表に示すように、odrv0.axis0.config.can の下にある一連の構成によってオン / オフを切り替えることができます（値 0 はオフ、その他の値は ms 単位の周期時間を意味します）：

メッセージ	odrivetool コンフィギュレーション	デフォルト
ハートビート	odrv0.axis0.config.can.heartbeat_rate_ms	100
Get_Encoder_Estimates(エンコーダー推定値)	odrv0.axis0.config.can.encoder_rate_ms	10
Get_Motor_Error	odrv0.axis0.config.can.motor_error_rate_ms	0
Get_Encoder_Error	odrv0.axis0.config.can.encoder_error_rate_ms。	0
Get_Controller_Error（コントローラエラー	odrv0.axis0.config.can.controller_error_rate_ms	0
Get_Sensorless_Error	odrv0.axis0.config.can.sensorless_error_rate_ms	0
Get_Encoder_Count	odrv0.axis0.config.can.encoder_count_rate_ms	0
Get_Iq	odrv0.axis0.config.can.iq_rate_ms	0
Get_Sensorless_Estimates	odrv0.axis0.config.can.sensorless_rate_ms	0
Get_Bus_Voltage_Current	odrv0.axis0.config.can.bus_vi_rate_ms	0

デフォルトでは、がオンになると最初の2つのサイクルメッセージがオンになるため、ユーザーがCANバスを監視すると、次の2種類のキャンセルが表示されます。

```
odrv0.axis0.config.can.heartbeat_rate_ms=0
odrv0.axis0.config.can.encoder_rate_ms=0
```

メッセージは設定された周期でブロードキャストされます。各メッセージの詳細

については、4.1.2 を参照してください。

4.2 CANOpenプロトコル

CANOpenは、オートメーション分野で使用される通信プロトコル仕様で、CiA301、CiA302、CiA402仕様に従って実装されています。本製品のCANOpenプロトコルを使用する前に、以下のodrivetoolコマンドで缶プロトコルをCANOpenに切り替えてください：

```
odrv0.can.config.protocol  = 2
```

4.2.1 デバイス記述ファイル（EDS）

EDSファイルには、デバイスの機能と通信パラメータが記述されています。CANOpen経由で製品と通信するには、製品のEDSファイルをインポートする必要があります。次のリンクからダウンロードしてください: https://bl.cyberbeast.cn/actuator/cb_0.1.eds

4.2.2 プロトコルフレームフォーマット

CANOpenは、標準CANに基づいています。 11ビットのIDは、4ビットのファンクションコードと7ビットのノードID（node_id）に分かれており、バス上で最大127ノードがサポートされています（0はブロードキャストに使用されます）。次の表は、フレーム・フォーマットを示しています：

データ・フィールド	COB-ID / CAN ID（11ビット）		データ（8 バイト）
セグメンテーション	ビット10~ ビット7	ビット6~ ビット0	バイト0~ バイト7
説明	機能 コード	ノードID	通信データ

ファンクションコードの表は以下の通り：

通信機能	COB-ID	説明
NMT	0x000	ネットワーク管理
SYNC	0x080	同期
EMCY	0x080+node_id	緊急時管理
TPDO1~TPDO4	0x180/0x280/0x380/0x480+node_id	リアルタイムプロセスデータ送信
RPDO1~RPDO4	0x200/0x300/0x400/0x500+node_id	リアルタイムプロセスデータ受信
TSDO	0x600+node_id	サービスデータ送信
RSDO	0x580+node_id	サービスデータ受信
NMT（心跳）	0x700+node_id	心跳

4.2.3 フレームメッセージ

CAN ID（COB-ID）により、フレームメッセージのフォーマットが異なります。

4.2.3.1 NMT（ネットワーク管理）

NMTは、バス上の全ノード、または特定のノードに対して状態変更コマンドを発行するために使用されます。 COB-IDは常に0であり、データは下表に示すように2バイトで構成されます：

COB-ID	データ バイト 0	データ・バイト0
データ・バイト 0 データ・バイト 1	ステータス・コマンド・ワード	ノードID

ステータス・コマンド・ワード

ステータス・コマンド・ワード	説明
0x01	ステータスが「動作可能」に変更された。
0x02	ステータスを「停止」に変更。
0x80	ステータスが「動作前」に変更された。
0x81	リセットノード "reset node"
0x82	通信のリセット "reset communication"

例えば、ドライブを接続した後、ユーザーはまず以下のNMTコマンドを送信し、バス上のすべてのノードを立ち上げます：

COB-ID	COB-ID
0x000	

4.2.3.2 同期

Sync 機能はバス上のすべてのノードを同期させるために使用され、主に PDO メッセージでノードを同期させるために使用される。COB-ID は 0x80 に固定され、データ・フィールドは空です。

例えば、ユーザが NMT 命令でバス上のノードを動作させた後、SYNC メッセージを送信してノードに PDO データの送信を開始させます。

4.2.3.3 PDO

PDOはリアルタイム・データ伝送のために用意された通信方式で、ノード・データを定期的に取得したい場合や、イベント・ドリブンでデータを報告したい場合に使用します。PDOはノードの状態が "operational "のときにのみ使用されるため、ユーザーは最初にNMTメッセージでノードの状態を変更する必要があります。

PDOメッセージのデータフィールドのフォーマットはマッピング構造によって決定され、これは標準化されていませんが、EDS文書によって指定されています。

4.2.3.4 SDO

SDO通信では、製品のオブジェクト・ディクショナリ・データにアクセスできます（オブジェクト・ディクショナリはEDSによって指定されます）。いわゆるオブジェクトディクショナリデータは、製品のパラメータ情報と理解することができる。一般的に、SDOは設定プロセスにのみ使用され、運用中のデータ交換には使用されません。リアルタイムの運用データ交換にはPDOを推奨する。

SDO は、読み取りと書き込みの 2 つのプロセスに分けられる。 以下の表は、書き込みプロセス（アップロードとも呼ばれる）を示している：

	COB-ID	COB-ID バイト0	データ バイト0 バイト1	データ バイト1 バイト2	データ バイト2 バイト3	バイト3 バイト4	バイト4 バイト5	データ バイト5 バイト6	データ バイト6 バイト7
データ バイト7	0x600+ ノードID	0x23	ノード ID: 0x23		サブインデックス	ノードID			
		0x27	サブインデックス・データ 0x27		データ	データ			/サブインデックスデータ
		0x2B	サブインデックス		サブインデックス	データ		/サブインデックスデータ	/
		0x2F	サブインデックスデータ / / 0x2F		サブインデックス	データ	/サブインデックスデータ	/	//
レスポンス	0x580+ ノードID	0x60	ノードID		サブインデックス	/サブインデックス	/	//	///
		0x80	インデックス		サブインデックス	中絶コード			

読み取り処理（ダウンロードともいう）：

	COB-ID	データ バイト0	データ Byte0 バイト1	データ バイト1 バイト2	データ バイト3	バイト3 バイト4	データ バイト4 バイト5	データ バイト5 バイト6	データ バイト6 バイト7
データバイト7	0x600+ ノードID	0x40	ノードID		サブインデックス	/サブインデックス	/	//	//
レスポンス	0x580+ ノードID	0x43	ノードID		サブインデックス	ノードID			
		0x47	サブインデックス・データ 0x47		データ	データ			/データ
		0x4B	サブインデックス		サブインデックス	データ		/サブインデックス・データ	/
		0x4F	サブインデックス・データ / / 0x4F		サブインデックス	データ	/サブインデックス・データ	/	//

		0x80	データ / / 0x80	サブインデックス	中絶コード

4.2.3.5 NMT（ハートビート）

ハートビート・メッセージはノードのステータスをバスに示すためだけに使用され、ノードが正常に動作していることを示します。フレーム・フォーマットは以下の通りです：

COB-ID	データ バイト0
0x700+node_id	ステータス

4.3 RS485 プロトコル (Modbus RTU)

バージョン3.10 以上のハードウェア・バージョンでは、通信インターフェースはRS485、パルス方向、PWM の両方をサポートしています。

```
odrv0.config.comm_intf_mux= 1
```

以下のコマンドで、通信インターフェイスが RS485 であることを確認します：

モーターウィザードでは、通信パラメータで物理インターフェイスを切り替えることもできます。

Modbus RTU ベースの RS485 最大通信レート 115200bps, デフォルトレート 115200bps.

4.3.1 Modbus RTU

工業規格に基づく Modbus RTU は、現在以下の機能コードをサポートしています：

機能コード	意味
0x04	単一または複数の入力レジスタを読み込む
0x03	複数の保持レジスタの読み出し
0x06	単一保持レジスタの書き込み
0x10	複数保持レジスタ書き込み

4.3.2 入力レジスタ

入力レジスタは読み取り専用であり、書き込み専用ではなく、エラーメッセージ、電圧／電流など、システムのいくつかのステータスまたは動的インジケータを表す：

- **0x01：モーター・ステータス**

ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	XX 04 00 01	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	状態0 状態1	XX XX

State0 は常に 0 で、State1 はモータのステータスコードです：

モータ状態コード	説明
1	アイドル
3	完全に較正済み
4	モーター較正
7	エンコーダの較正
8	クローズドループ動作
11	ゼロリターン

- **0x02：例外フラグ**

ホストからの要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 01	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	フラグ0 フラグ1	XX XX

Flag0は常に0であり、Flag1は例外フラグであり、各ビットは、対応する例外を表すために1であり、例外は次のように表されます。

例外は次のように表される：

bit7~bit5	bit4	bit3	bit2	bit1	bit1
意味なし	ドライバ例外	システム異常	制御異常	エンコーダ異常	モータ異常

- **0x03：ドライブ例外コード 1**
- **0x04：ドライブ例外コード 2**

ドライブ例外コードは32ビットの符号なし整数で、ドライブ例外コード1が上位16ビット、ドライブ例外コード2が下位16ビットである。ユーザーはレジスタ・アドレス 0x03 にのみアクセスして、両方のレジスタ・データを同時に取得し、完全なドライ ブ例外コードを得ることができます。どちらのレジスタも個別にアクセスすることはできません。

ホストからの要求：

アドレス	ファンクション・コード	レジスタ・アドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 03	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 はドライブ例外コードを構成します。例外コードの表は 5.2 節を参照してください。

- **0x05: モータ例外コード 1**
- **0x06: モータ例外コード 2**
- **0x07: モータ例外コード 3**
- **0x08：モーター例外コード 4**

モーター例外コードは 64 ビットの符号なし整数で、ビット 63 から 48 がモーター例外コード 1、47 から 32 がモーター例外コード 2、47 から 32 がモーター例外コード 3 です。

モーター例外コード1は63～48ビット、モーター例外コード2は47～32ビット、モーター例外コード3は31～16ビット、モーター例外コード4は15～0ビットです。ユーザはレジスタ・アドレス 0x05 にのみアクセスし、4 つのレジスタすべてに同時にアクセスすることで、完全なモータ例外コードを得ることができます。どのレジスタも個別にアクセスすることはできません。

ホストからの要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 04	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	8バイト	2バイト
XX	XX	XX 04	E0 E1 E2 E3 E4 E5 E6 E7	XX XX

E0 E1 E2 E3 E4 E5 E6 E7 はモータ例外コードを構成します。例外コード表は 5.2 節を参照してください。

- **0x09：エンコーダ例外コード 1**
- **0x0A：エンコーダ例外コード 2**

エンコーダ例外コードは 32 ビットの符号なし整数で、 エンコーダ例外コード 1 が上位 16 ビット、 エンコーダ例外コード 2 が下位 16 ビットです。ユーザーはレジスタ・アドレス0x09にのみアクセスし、両方のレジスタに同時にアクセスすることで、完全なエンコーダ例外コードを得ることができます。どちらのレジスタも個別にアクセスすることはできません。

ホストからの要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 はエンコーダ例外コードを構成します。例外コード表は 5.2 節を参照してください。

- **0x0B：制御例外コード 1**
- **0x0C：制御例外コード 2**

制御例外コードは32ビットの符号なし整数で、制御例外コード1が上位16ビット、制御例外コード2が下位16ビットです。ユーザーはレジスタ・アドレス0x0Bにのみアクセスし、両方のレジスタに同時にアクセスすることで、完全な制御例外コードを得ることができる。どちらのレジスタも個別にアクセスすることはできない。

ホストからの要求：

アドレス	機能コード	レジスタ・アドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 0B	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 は制御例外コードを構成します。例外コード表は 5.2 節を参照してください。

- **0x0D：システム例外コード 1**
- **0x0E: システム例外コード 2**

システム例外コードは32ビットの符号なし整数で、システム例外コード1が上位16ビット、システム例外コード2が下位16ビットである。ユーザーは、レジスタ・アドレス0x0Dにのみアクセスし、両方のレジスタに同時にアクセスすることで、完全なシステム例外コードを得ることができる。どちらのレジスタも個別にアクセスすることはできない。

ホストからの要求：

アドレス	機能コード	レジスタ・アドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	E0 E1 E2 E3	XX XX

E0 E1 E2 E3 はシステム例外コードを構成します。例外コード表は 5.2 節を参照してください。

- **0x0F: 位置フィードバック 1**
- **0x10: 位置フィードバック 2**

位置フィードバックは 32 ビットの浮動小数点数で、位置フィードバック 1 は上位 16 ビット、位置フィードバック 2 は下位 16 ビットです。ユーザは、レジスタアドレス 0x0F にアクセスし、両方のレジスタから同時にデータを取得することによってのみ、完全な位置フィードバックを得ることができる。どちらのレジスタも個別にアクセスすることはできない。

ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	P0 P1 P2 P3	XX XX

P0 P1 P2 P3 は位置フィードバックを形成し、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x11: 速度フィードバック 1**
- **0x12: 速度フィードバック 2**

速度フィードバックは 32 ビットの浮動小数点数で、速度フィードバック 1 は上位 16 ビット、速度フィードバック 2 は下位 16 ビットです。ユーザは、完全な速度フィードバックを得るためにレジスタ・アドレス 0x11 にアクセスすることによってのみ、両方のレジスタにアクセスできます。どちらのレジスタも個別にアクセスすることはできません。

ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	XX 04 00 11	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3 は速度フィードバックで構成され、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x13: バス電圧フィードバック 1**
- **0x14: バス電圧フィードバック 2**

バス電圧は 32 ビット浮動小数点数で、バス電圧 1 が上位 16 ビット、バス電圧 2 が下位 16 ビットです。ユーザーは、レジスタ・アドレス 0x13 にアクセスし、両方のレジスタを同時に取得することによってのみ、完全なバス電圧にアクセスすることができます。どちらのレジスタも個別にアクセスすることはできません。

ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 02	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	V0 V1 V2 V3	XX XX

V0 V1 V2 V3 はバス電圧で構成され、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x15: バス電流フィードバック 1**
- **0x16: バス電流フィードバック 2**

バス電流フィードバックは 32 ビットの浮動小数点数で、バス電流フィードバック 1 が上位 16 ビット、バス電流フィードバック 2

が下位 16 ビットです。

バス電流フィードバック 1 は上位 16 ビット、バス電流フィードバック 2 は下位 16 ビットです。ユーザーは、レジスタ・アドレス 0x15 にアクセスし、両方のレジスタから同時にデータを取得することによってのみ、完全なバス電流フィードバックにアクセスすることができます。どちらのレジスタも個別にアクセスすることはできません。

ホスト・リクエスト：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 04	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3 はバス電流フィードバックを形成し、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x17: Q軸電流目標値 1**
- **0x18: Q軸電流目標値 2**

Q 軸電流目標は 32 ビットの浮動小数点数で、Q 軸電流目標 1 は上位 16 ビット、Q 軸電流目標 2 は下位 16 ビットです。完全な Q 軸電流目標は、レジスタ・アドレス 0x17 にアクセスし、両方のレジスタを同時に取得することによってのみアク セスできる。どちらのレジスタも個別にアクセスすることはできない。

ホストからの要求：

アドレス	ファンクション・コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 17	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3 は Q 軸電流目標値を形成し、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x19: Q軸電流フィードバック1**
- **0x1A: Q 軸電流フィードバック 2**

Q 軸電流フィードバックは 32 ビットの浮動小数点数で、Q 軸電流フィードバック 1 は上位 16 ビット、Q 軸電流フィードバック 2 は下位 16 ビットです。ユーザは、レジスタアドレス 0x19 にアクセスして両方のレジスタから同時にデータを取得することによってのみ、完全な Q 軸電流フィードバックにアクセスできます。どちらのレジスタも個別にアクセスすることはできない。

ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 19	XX 04 00 19	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 04	I0 I1 I2 I3	XX XX

I0 I1 I2 I3はQ軸電流フィードバックを構成し、32ビット浮動小数点数はIEEE 754規格に従ってエンコードされる（エンコードはウェブサイトhttps://www.h-schmidt.net/FloatConverter/IEEE754.html でテスト可能）。

- **0x31：ランダムエンドポイントアクセス 戻り値 1**

現時点ではサポートされていない。

- **0x32：ランダムエンドポイントアクセス戻り値 2**

現時点ではサポートされていない。

4.3.3 保持レジスタ

現在サポートされているレジスタとその機能は以下の通り：

- **0x61：緊急停止**

このレジスタに任意の値を書き込むと、モーターが緊急停止します。

ホストからの要求

アドレス	機能 コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 61	XX XX	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 61	XX XX	XX XX

- **0x62：ノード ID**

ファンクションコード 06 でこのノードにノード ID を書き込むか、ファンクションコード 03 で現在の

ノード ID を読み込む：

アドレス	機能コード	レジスタアドレス	レジスタアドレス	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト

XX	XX	00 62	ID0 ID1	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 62	ID0 ID1	XX XX

ここで、ID0 と ID1 は新しいノード ID の上位 16 ビットと下位 16 ビット

です。現在のノード ID の読み取り要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 62	00 01	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	ID0 ID1	XX XX

- **0x63：状態制御**

06 のファンクションコードでこのノードにモータの状態を書き込む（モータの

状態を変更する）。ホストからの要求

アドレス	ファンクションコード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 63	S0 S1	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 63	S0 S1	XX XX

ここで、S0 と S1 はモータの状態の上位 16 ビットと下位 16 ビットである。ステータスの定義は以下の通り：

モータ・ステータス・コード	説明
1 アイドル	アイドル
3	完全に較正済み
4	モーター較正
7	エンコーダの較正
8	クローズドループ動作
11	ゼロリターン

例えば、モータ状態を 8（クローズドループ動作）に設定することは、通常の意味での「モータのス イッチオン」を意味し、モータ状態を 1（アイドル状態）に設定することは、通常の意味での「モータのス イッチオフ」を意味する。

- **0x64：制御モード**

ユーザは、06 ファンクション・コードを使用してこのレジスタの値を変更し、 トルク制御、速度制御、または位置制御モードに入ることができます。

現在の制御モードを読み出すには 03 ファンクション・コードを使用します。

制御モード変更要求

アドレス	ファンクションコード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 64	M0 M1	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 64	M0 M1	XX XX

制御モード読み出し要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 64	00 01	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	M0 M1	XX XX

ここで、M0 と M1 は制御モードの上位 16 ビットと下位 16 ビットである。制御モードを以下に定義する：

制御モード	制御モード
1	トルク制御
2	速度制御
3	位置制御

➢ **0x65：入力モード**

ユーザはこのレジスタの値をファンクション・コード 06 で変更し、目標値に対する異なる入力モード（例：ダイレクト、 フィルタなど）を選択することができ、またファンクション・コード 03 で現在の入力モードを読み出すことができます。

ユーザはこのレジスタの値をファンクション・コード 06 で変更し、直接入力、 フィルタ入力などの異なる目標値入力モードを選択するか、ファンクション・コード 03 を使用して現在の入力モードを読み出すことができます。

入力モード変更要求：

アドレス	ファンクションコード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 65	M0 M1	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 65	M0 M1	XX XX

入力モード読み出し要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 65	00 01	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	03	02	M0 M1	XX XX

ここで、M0 と M1 は入力モードの上位 16 ビットと下位 16 ビットである。入力モードは以下のように定義される：

制御モード	制御モード	詳細説明
1	直接入力	ユーザが入力した目標値をそのままドライブの目標値とする。
2	速度ランプ	ドライブの速度目標値に一定時間内に一定の勾配で到達させる。 ユーザが入力した速度目標値は、速度制御のみに使用されます。
3	位置フィルタ	ドライブの位置目標値は、ユーザが入力した位置目標値をフィルタリングした結果です。 位置目標値は、ユーザが入力した位置目標値をフィルタリングした値であり、位置制御のみに使用される。
5	台形カーブ	ドライブは、ユーザが入力した目標値に到達するように、台形状の速度プロファイルでモータを駆動します。 台形の上向き勾配（加速度）、下向き勾配（減速度）、滑走速度を設定することができ、以下のレジスタ 0x70～0x75 を参照すること。
6	トルクランプ	ドライブの目標トルク値を、一定の勾配に従って一定時間内にユーザが入力した目標トルク値に到達させるもので、トルク制御のみに使用する。

- **0x66：位置制御目標 1**
- **0x67：位置制御目標 2**

位置制御目標は 32 ビットの浮動小数点数で、上位 16 ビットを位置制御目標 1、下位 16 ビットを位置制御目標 2 とする。位置制御目標1は16ビット、位置制御目標2は16ビット。ユーザーは、レジスタ・アドレス0x66にのみアクセスし、完全な位置制御目標を取得/変更するために、両方のレジスタを同時に読み書きすることができる。どちらのレジスタも別々にアクセスすることはできません。

位置制御目標の変更／更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 66	00 02	04	P0 P1 P2 P3	XX XX

モーター応答

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 66	00 02	XX XX

位置制御目標読出要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト

XX	XX	00 66	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	P0 P1 P2 P3	XX XX

ここで、P0 P1 P2 P3 は位置制御ターゲットの上位から下位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト h[illegible]テストできます）。

- **0x68: 速度制御目標 1**
- **0x69: 速度制御目標 2**

速度制御目標は 32 ビットの浮動小数点数で、速度制御目標 1 は上位 16 ビット、速度制御目標 2 は下位 16 ビットです。
速度制御目標 1 は上位 16 ビット、速度制御目標 2 は下位 16 ビットである。ユーザは、レジスタ・アドレス 0x68 にアクセスし、両方のレジスタを同時に読み書きして、完全な速度制 御ターゲットを取得／変更することしかできない。どちらのレジスタも個別にアクセスすることはできません。

速度制御ターゲットの変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 68	00 02	04	V0 V1 V2 V3	XX XX

モーター応答

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 68	00 02	XX XX

速度制御対象読み出し要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 68	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	V0 V1 V2 V3	XX XX

ここで、V0 V1 V2 V3 は速度制御ターゲットの上位から下位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされています（エンコードはウェブサイト h[illegible]テストできます）。

- **0x6A：トルク制御目標 1**
- **0x6B: トルク制御目標 2**

トルク制御目標は32ビットの浮動小数点数で、トルク制御目標1が上位16ビット、トルク制御目標2が下位16ビットです。
トルク制御目標 1 は上位 16 ビット、トルク制御目標 2 は下位 16 ビットである。ユーザーは、完全なトルク制御目標を取得/変更するために、レジスタ・アドレス 0x6A にのみアクセスし、両方のレジスタを同時に読み書きすることができます。どちらのレジスタも別々にアクセスすることはできません。

トルク制御目標の変更/更新要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 6A	00 02	04	T0 T1 T2 T3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 10	00 02	XX XX

トルク制御目標値読み出し要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 6A	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	T0 T1 T2 T3	XX XX

ここで、T0 T1 T2 T3 はトルク制御ターゲットの上位から下位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト [illegible] テストできます）。

- **0x6C: 速度制限 1**
- **0x6D: 速度制限 2**

速度制限は 32 ビットの浮動小数点数で、速度制限 1 は上位 16 ビット、速度制限 2 は下位 16 ビットです。ユーザーはレジスタ・アドレス 0x6C にのみアクセスし、両方のレジスタを同時にリード/ライトして、完全な速度制限を取得/変更することができる。どちらのレジスタも個別にアクセスすることはできない。

速度制限の変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	XX 10	00 6C	04	V0 V1 V2 V3	XX XX

モーター応答

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 10	00 02	XX XX

制限速度の読み出し要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 6C	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	V0 V1 V2 V3	XX XX

ここで、V0 V1 V2 V3 は速度制限の上位から下位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされています（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x6E: 電流制限 1**
- **0x6F: 電流制限 2**

電流制限は32ビットの浮動小数点数で、電流制限1は上位16ビット、電流制限2は下位16ビットである。ユーザーは、レジスタ・アドレス 0x6E にアクセスし、両方のレジスタを同時に読み書きすることによってのみ、完全な電流制限にアクセス/変更することができる。どちらのレジスタも個別にアクセスすることはできない。

電流制限の変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	XX 10	00 6E	04	I0 I1 I2 I3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット

1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 10	00 02	XX XX

電流制限値読み出し要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 6E	00 02	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	I0 I1 I2 I3	XX XX

ここで、I0 I1 I2 I3 は電流制限の上位から下位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされています（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x70：ラダー位置制御速度制限1**
- **0x71：ラダー位置制御速度制限 2**

台形位置制御速度制限は32ビットの浮動小数点数で、台形位置制御速度制限1は上位16ビット、台形位置制御速度制限2は下位16ビットです。

ラダー位置制御速度制限1は上位16ビット、ラダー位置制御速度制限2は下位16ビットです。完全なラダー位置制御速度制限は、レジスタ・アドレス0x70にアクセスし、両方のレジスタを同時に読み書きすることによってのみ取得/変更できます。どちらのレジスタも個別にアクセスすることはできません。

ラダー位置制御速度制限の変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 70	00 02	04	V0 V1 V2 V3	XX XX

モーター応答

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 70	00 02	XX XX

ラダー位置制御速度制限要求を読み出します：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	03	00 70	00 02	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	V0 V1 V2 V3	XX XX

ここで、V0 V1 V2 V3 は台形位置制御速度制限の高位から低位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされています（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x72：台形位置制御加速度リミット 1**
- **0x73：台形位置制御加速度リミット 2**

台形位置制御加速度リミットは 32 ビットの浮動小数点数で、台形位置制御加速度リミ ット 1 は上位 16 ビットである。台形位置制御加速度リミット 2 は下位 16 ビットです。ユーザは、レジスタ・アドレス0x72にアクセスし、両方のレジスタを同時に読み書きすることによってのみ、完全な台形位置制御加速度制限にアクセス／変更することができる。どちらのレジスタも個別にアクセスすることはできません。

ラダー位置制御加速リミットの変更 / 更新リクエスト：

アドレス	ファンクションコード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 72	00 02	04	A0 A1 A2 A3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 72	00 02	XX XX

ラダー位置制御加速度制限要求を読み出す：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 72	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	A0 A1 A2 A3	XX XX

ここで、A0 A1 A2 A3 は台形位置制御の減速リミットを高から低へ 4 バイトで表し、32 ビット浮動小数点数は IEEE 754 標準規格に従ってエンコードされる（ウェブ https://www.h-schmidt.net/FloatConverter/IEEE754.html測试编码 で入手可能）。

- **0x74：台形位置制御減速度制限 1**
- **0x75：台形位置制御減速度制限 2**

台形位置制御減速度リミットは 32 ビットの浮動小数点数で、台形位置制御減速度リミット 1 は上位 16 ビットである。台形位置制御減速度制限2は下位16ビットである。完全な台形位置制御減速リミットは、レジスタ・アドレス 0x74 にアクセスし、両方のレジスタを同時に読み書きすることによってのみ取得／変更できる。どちらのレジスタも別々にアクセスすることはできません。

ラダー位置制御減速リミットの変更 / 更新要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	XX 10	00 02	04	D0 D1 D2 D3	XX XX

モーター応答

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	XX 10	00 02	XX XX

ラダー位置制御減速度制限要求を読み出します：

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 74	00 02	XX XX

モーター応答：

アドレス	機能コード	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	D0 D1 D2 D3	XX XX

ここで、D0 D1 D2 D3 は台形位置制御の減速限界の高位から低位までの 4 バイトで、32 ビット浮動小数点数は IEEE 754 規格に従ってエンコードされます（ウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html測试编码 で入手可能）。

- **0x76：イナーシャ 1**
- **0x77: 慣性 2**

イナーシャは 32 ビット浮動小数点数で、イナーシャ 1 が上位 16 ビット、イナーシャ 2 が下位 16 ビットである。ユーザはレジスタ・アドレス 0x76 にのみアクセスし、完全なイナーシャを取得／変更するために両方のレジスタを同時にリード／ライトすることができる。どちらのレジスタも別々にアクセスすることはできない。

イナーシャの変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェック デジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	10	00 76	00 02	04	I0 I1 I2 I3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 76	00 02	XX XX

慣性モーメントの読み出し要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 76	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	I0 I1 I2 I3	XX XX

ここで、I0 I1 I2 I3 は、32 ビット浮動小数点数が IEEE 754 標準に従ってエンコードされた、高から低の 4 バイトのイナーシャです（エンコードはウェブサイト https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x78: 位置ゲイン 1**
- **0x79：ポジションゲイン2**

位置ゲインは32ビットの浮動小数点数で、位置ゲイン1が上位16ビット、位置ゲイン2が下位16ビットである。ユーザーは、レジスタ・アドレス0x78にアクセスし、両方のレジスタを同時に読み書きすることによってのみ、完全な位置ゲインにアクセス/変更することができる。どちらのレジスタも個別にアクセスすることはできません。

位置ゲインの変更/更新要求：

アドレス	機能 コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェック デジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 78	00 78 00 02	04	G0 G1 G2 G3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 78	00 02	XX XX

位置ゲイン読み取り要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 78	00 02	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	G0 G1 G2 G3	XX XX

ここで、G0 G1 G2 G3 は、高から低への位置ゲインの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードは https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x7A：速度ゲイン1**
- **0x7B: 速度ゲイン 2**

速度ゲインは32ビットの浮動小数点数で、速度ゲイン1は上位16ビット、速度ゲイン2は下位16ビットです。ユーザーは、レジスタ・アドレス 0x7A にのみアクセスし、両方のレジスタを同時にリード / ライトして、全速度ゲインを取得 / 変更することができます。どちらのレジスタも個別にアクセスすることはできません。

速度ゲインの変更/更新要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェック デジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	00 7A	00 02	04	G0 G1 G2 G3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット

1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7A	00 02	XX XX

読み出し速度ゲイン要求

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	03	00 7A	00 02	XX XX

モーター応答

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	G0 G1 G2 G3	XX XX

ここで、G0 G1 G2 G3 は速度ゲインのハイからローまでの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードは https://www.h-schmidt.net/FloatConverter/IEEE754.html でテストできます）。

- **0x7C: 速度積分ゲイン 1**
- **0x7D：速度積分ゲイン 2**

速度積分ゲインは 32 ビットの浮動小数点数で、速度積分ゲイン 1 は上位 16 ビット、速度積分ゲイン 2 は下位 16 ビットです。速度積分ゲインは 32 ビット浮動小数点数で、速度積分ゲイン 1 が上位 16 ビット、速度積分ゲイン 2 が下位 16 ビットです。ユーザは、全速度積分ゲインを取得/変更するために、レジスタ・アドレス 0x7C にアクセスし、両方のレジスタを同時にリード/ライトすることしかできません。どちらのレジスタも個別にアクセスすることはできません。

速度積分ゲインの変更/更新要求：

アドレス	機能コード	レジスタアドレス	レジスタ数	バイト数	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	1バイト	4バイト	2バイト
XX	XX	XX 10	00 02	04	G0 G1 G2 G3	XX XX

モーター応答

アドレス	機能コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7C	00 02	XX XX

速度積分ゲイン読み出し要求

アドレス	機能 コード	レジスタアドレス	レジスタ数	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7C	00 02	XX XX

モーター応答：

アドレス	機能コード	バイトサイズ	レジスタ値	チェックデジット
1バイト	1バイト	1バイト	2バイト	2バイト
XX	XX	XX 03	G0 G1 G2 G3	XX XX

ここで、G0 G1 G2 G3 は速度積分ゲインのハイからローまでの 4 バイトで、32 ビット浮動小数点数は IEEE 754 標準に従ってエンコードされます（エンコードはウェブサイト [illegible] でテストできます）。

➢ **0x7E: 再起動**

このレジスタに任意の値を書き込むと、モーター・ドライブが再起動しま

す。ホストからの要求：

アドレス	機能 コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7E	XX XX	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7E	XX XX	XX XX

➢ **0x7F：例外クリア**

このレジスタに任意の値を書き込むと、すべてのエラーと例外がクリアされ

る。ホストからの要求：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7F	XX XX	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 7F	XX XX	XX XX

➢ **0x80: コンフィギュレーションの保存**

このレジスタに任意の値を書き込むと、現在のすべてのパラメータがドライブ・フラッシュに永久保存されます。ホストからの要求：

アドレス	機能 コード	レジスタ・アドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 80	XX XX	XX XX

モーター応答：

アドレス	機能コード	レジスタアドレス	レジスタ値	チェックデジット
1バイト	1バイト	2バイト	2バイト	2バイト
XX	XX	00 80	XX XX	XX XX

➢ **0xA1：ランダムエンドポイントアクセスエンドポイント番号**

現時点ではサポートされていない。

➢ **0xA2：ランダム・エンドポイント・アクセス書き込み値 1**

現時点ではサポートされていません。

➢ **0xA3：ランダム・エンドポイント・アクセス書き込み値 2**

サポートされていません。

4.3.4 Modbus プロトコル

モーターのアドレスを1とする。

1) 実用：速度制御

フレームの送信順序は以下の通り：

フレームデータ	レジスタ	レジスタ
01 06 00 64 00 02 xx xx	保持レジスタ 0x64	制御モードを 2（速度制御）に設定
01 06 00 65 00 02 xx xx	保持レジスタ 0x65	入力モードを 2（速度ランプ）に設定する
01 06 00 63 00 08 xx xx	保持レジスタ 0x63	モータの状態を 8（クローズドループ動作）に変更します。
01 10 00 68 00 02 04 40 13 33 33 XX XX	保持レジスタ 0x68	目標速度を 2.3r/s に設定し、32 ビット浮動小数点数 40 13 33 33 に変換します。 ビット浮動小数点数を 40 13 33 33 に変換。

2) 実践：位置制御

フレーム送信のシーケンスは以下の通りである：

フレーム・データ	レジスタ	レジスタ
01 06 00 64 00 03 xx xx	保持レジスタ 0x64	制御モードを 3（位置制御）に設定
01 06 00 65 00 03 xx xx	保持レジスタ 0x65	入力モードを 3 に設定（位置フィルタリング）
01 06 00 63 00 08 xx xx	保持レジスタ 0x63	モータ状態を 8（クローズドループ動作）に変更する。
01 10 00 68 00 02 04 40 49 0F F9 XX XX	保持レジスタ 0x68	目標速度を 3.1416r に設定、32 ビット浮動小数点数 40 49 0F F9 XX XX に変換。 ビット浮動小数点数 40 49 0F F9 に変換。

4.4 SDK

4.4.1 C/C++ SDK

CAN Simpleプロトコルに基づき、以下のC/C++ SDKをダウンロードして統合することができます。https://gitee.com/cyberbeast/mwmotorsdk。統合テストについては、上記のオープンソースウェブサイトの詳細なテスト例を参照してください。

4.4.2 Python SDK

3.1節にあるように、まずodrivetoolをインストールしてください（pip install -upgrade odrive）。3.1.8節を参照し、そのサブセクションで説明されているすべてのコマンドでpython開発を行なってください。

以下は3つの例です：

1) 実用：パワーアップキャリブレーション

```
インポート odrive
インポート時間


odrv0= odrive.find_any()
odrive.utils.dump_errors(odrv0) odrv0.clear_errors()
odrv0.axis0.requested_state=odrive.utils.AxisState.MOTOR_CALIBRATION time.sleep(5)
while (odrv0.axis0.current_state!=1)：
    time.sleep(0.5) odrive.utils.dump_errors(odrv0)
odrv0.axis0.requested_state=odrive.utils.AxisState.ENCODER_OFFSET_CALI BRATION
time.sleep(6)
while (odrv0.axis0.current_state!=1): time.sleep(0.5)
odrv0.axis0.motor.config.pre_calibrated=1 odrv0.axis0.encoder.config.pre_calibrated=1
odrv0.save_configuration()
```

2) 実践：スピードコントロール

```
インポートodrive
インポート時間


odrv0= odrive.find_any()
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.VE LOCITY_CONTROL
odrv0.axis0.controller.config.input_mode=odrive.utils.InputMode.VEL_RA MP
odrv0.axis0.controller.config.vel_ramp_rate=50
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL odrv0.axis0.コントローラ
.input_vel=15
odrive.utils.dump_errors(odrv0) time.sleep(5)
odrv0.axis0.controller.input_vel=0
```

3) 実践：位置制御

```
インポート odrive


odrv0= odrive.find_any()
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.PO    SITION_CONTROL
odrv0.axis0.controller.config.input_mode=odrive.utils.InputMode.POS_FI    LTER
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL。

odrv0.axis0.controller.input_pos=10
```

4) 実用：データ収集

PythonSDKは強力なデータキャプチャ機能を統合しており、簡単なスクリプトで大量の運転データをキャプチャすることができます。

以下のコードは、前回の位置制御の例をベースに、リアルタイムの位置・電流データキャプチャ機能を追加したものです。

```
インポート odrive
npとしてnumpyをインポート
する。

odrv0= odrive.find_any()
cap=  odrive.utils.BulkCapture(lambda:[odrv0.axis0.motor.current_control.Iq_
measured,odrv0.axis0.encoder.pos_estimate],data_rate=500、 duration=2.5)
odrv0.axis0.controller.config.control_mode=odrive.utils.ControlMode.PO    SITION_CONTROL
odrv0.axis0.controller.config.input_mode=odrive.utils.InputMode.POS_FI    LTER
odrv0.axis0.requested_state=odrive.utils.AxisState.CLOSED_LOOP_CONTROL。

odrv0.axis0.controller.input_pos=10 np.savetxt("d:/test.csv",cap.data,delimiter=',')
```

データをcsvファイルとして保存します。

BulkCaptureステートメントでは、data_rateはサンプリング周波数（hz）、durationはサンプリング時間（秒）を表し、ラムダ式はデータ解析を容易にするために任意の数学演算を挿入するために使用できます。

4.4.3 Arduino SDK

Arduinoを使用して、CANバス経由でモーターを制御できます。基本的なプロトコルは4.1で説明します。互換性のあるハードウェア/ライブラリ

✓ Arduino UNO R4 Minima、Arduino UNO R4 WIFIなど、CANインターフェースを内蔵したArduino。

✓ CANインターフェースを内蔵したTeensyボードには、適合したFlexCAN_T4ライブラリを使用してアクセスできます（Teensy 4.0およびTeensy 4.1）。

✓ その他のArduino互換ボードには、MCP2515ベースのCAN拡張ボードを使用してアクセスできます

。 以下は、Arduinoの位置制御コマンドに応答するモーターの設定方法の例です：

➢ モーターの設定

3.1.3の基本設定に加えて、制御帯域幅が20 rad/sになるように制御を設定します（Arduino Unoの送信速度は制限されているため、制御帯域幅はあまり大きくする必要はありませんが、より高速なArduinoを使用する場合は大きくすることができます）：

➢ CANの設定

以下の説明に従ってCANを設定します：

➢ ODriveArduinoライブラリをインストールする。

以下の手順に従って、OdriveArduinoライブラリをインストールしてください（Arduino IDEがインストール済みであることが前提です）：

1) Arduino IDEを開く

2) Sketch -> Include Library -> Manage Libraries

3) ODriveArduino "を入力して検索します。

4) 見つかったODriveArduinoライブラリをクリックしてインストールします。

➢ Arduinoソースコード

```
#include <Arduino.h> #include
"ODriveCAN.h"


// この例のドキュメントは、ここで見ることができます。
// https://docs.odriverobotics.com/v/latest/guides/arduino-can-guide.html



/* スケッチ例の構成                                                                                */
                                                ------------------------------------------------------------------------
// CANバスのボーレート。 バス上のすべてのデバイスでこれが一致することを確認してください #define
CAN_BAUDRATE 500000


// odrv0用のODriveノードID #define
ODRV0_NODE_ID 0


// ハードウェアに対応する行の下のコメントを外します。
```

```
// ハードウェアのセットアップの詳細については、「ボード固有の設定」も参照してください。


// #define IS_TEENSY_BUILTIN // CANインターフェースを内蔵したTeensyボード (Teensy 4.1など)。 使用するインターフェースを選択するには、以下を参照してください。
// #define IS_ARDUINO_BUILTIN // CANインターフェースを内蔵したArduinoボード (Arduino Uno R4 Minimaなど)。
// #define IS_MCP2515 // MCP2515ベースの拡張モジュールを外付けしたボード。 モジュールを設定するには以下を参照してください。



/* ボード固有のインクルード                                                                                    /* ボード固有のインクルード                    ----------------------------------------------------------------------------------------------


#if defined(IS_TEENSY_BUILTIN)+ defined(IS_ARDUINO_BUILTIN)+  defined(IS_MCP2515)  ！= defined(IS_MCP2515)  ！
#warning "このファイルの先頭で、ハードウェアオプションを正確に1つ選択してください。"


#if CAN_HOWMANY> 0|| CANFD_HOWMANY>0
#define IS_ARDUINO_BUILTIN
#警告 "HardwareCANを使用していると推測されます" #else
#エラー "ハードウェアのバージョンを推測できません"
#endif

#endif


#ifdef IS_ARDUINO_BUILTIN
// https://github.com/arduino/ArduinoCore-API/blob/master/api/HardwareCAN.h を。参照
// および
https://github.com/arduino/ArduinoCore-renesas/tree/main/libraries/Arduino_CAN


#include <Arduino_CAN.h> #include
<ODriveHardwareCAN.hpp> #endif //
IS_ARDUINO_BUILTIN


#ifdef IS_MCP2515
// https://github.com/sandeepmistry/arduino-CAN/ を参照 #include "MCP2515.h"
#include "ODriveMCPCAN.hpp" #endif //
IS_MCP2515


#ifdef IS_TEENSY_BUILTIN
// https://github.com/tonton81/FlexCAN_T4 を参照。
// https://github.com/tonton81/FlexCAN_T4.git を /src にクローンする #include
<FlexCAN_T4.h
```

```
#include "ODriveFlexCAN.hpp"

struct ODriveStatus; // teensyコンパイルエ ラ ー を 防ぐためのハック #endif //

IS_TEENSY_BUILTIN




/* ボ ー ド固有の設定                                                                                                    /* ボ
ー ド固有の設定 /* Teensy */              ----------------------------------------------------------------------------------------------------



/* Teensy */

#ifdef IS_TEENSY_BUILTIN


FlexCAN_T4<CAN1, RX_SIZE_256, TX_SIZE_16> can_intf；

bool setupCan() { can_intf.begin().
    can_intf.setBaudRate(CAN_BAUDRATE); can_intf.setMaxMB(16);
    can_intf.enableFIFO(); can_intf.enableFIFOInterrupt();
    can_intf.onReceive().can_intf.onReceive( onCanMessage);
    return true ；
}

#endif // IS_TEENSY_BUILTIN



/* MCP2515 ベースの拡張モジュール -*/ #ifdef IS_MCP2515


MCP2515Class& can_intf= CAN; // MCP2515で使用されるチップ・セレクト・ピン。


// MCP2515で使用されるチップセレクトピン #define
MCP2515_CS 10


// MCP2515で使用する割り込みピン。
// 注意: すべてのArduinoピンが割り込み可能なわけではありません。


// MCP2515ブレークアウトボードの水晶発振器の周波数。
// 一般的な値は 16MHz、12MHz、8MHz です。
```

```
#定義 MCP2515_CLK_HZ 8000000



static inline void receiveCallback(int packet_size) { if (packet_size> 8) { // サ
    ポートされていません。
        return; // サポートされていません
    }
    CanMsg msg= { .id= (unsigned int)CAN.packetId(), .len= (uint8_t)packet_size}; CAN.readBytes(msg.buffer, packet_size);
    onCanMessage(msg.buffer, packet_size); return; // サポートされていません }.
    onCanMessage(msg)；
}

ブール setupCan() {
    CAN.setPins(MCP2515_CS, MCP2515_INT); CAN.setClockFrequency(); CAN.readBytes(msg.buffer, packet_size);
    onCanMessage(msg); } // CANバス・インターフェースの設定と初期化
    CAN.setClockFrequency(MCP2515_CLK_HZ); if
    (!CAN.begin(CAN_BAUDRATE)) {
        return false; }.
    }

    CAN.onReceive(receiveCallback); return true; }.
}

#endif // IS_MCP2515



/* CAN内蔵Arduino */ #ifdef IS_ARDUINO_BUILTIN


HardwareCAN& can_intf= CAN；

ブール setupCan() {
    return can_intf.begin((CanBitRate)CAN_BAUDRATE)；
}

#}



/* スケッチ例                                                                                                    */

// ODriveオブジェクトのインスタンス化
ODriveCAN odrv0(wrap_can_intf(can_intf), ODRV0_NODE_ID); // 標準CANメッセージID



                        ----------------------------------------------------------------------------------------------------
```

```
ODriveCAN* odrives[]= {&odrv0}; // すべてのODriveCANインスタンスがここで説明されていることを確認してください。


struct ODriveUserData { heartbeat_msg_t last_heartbeat;
    bool received_heartbeat= false；
    Get_Encoder_Estimates_msg_t last_feedback; bool received_feedback=
    false；
}; // アプリケーション固有のデータを保持する。


// ODriveUserData odrv0_user_data；


// ODrive からハートビートメッセージが到着する度に呼び出される void
onHeartbeat(Heartbeat_msg_t& msg, void* user_data) { // ODrive からハートビートメッセージが到着
する度に呼び出される
    ODriveUserData* odrv_user_data= static_cast<ODriveUserData*>(user_data); odrv_user_data->last_heartbeat= msg；
    odrv_user_data->received_heartbeat= true；
}


// ODriveからフィードバックメッセージが到着するたびに呼び出される
void onFeedback(Get_Encoder_Estimates_msg_t& msg, void* user_data) { ODriveUserData* odrv_user_data=
    static_cast<ODriveUserData*> (user_data); odrv_user_data -> received_heartbeat (user_data); } // ODriveからフィード
    バックメッセージが到着するたびに呼び出される。(user_data); odrv_user_data->last_feedback= msg；
    odrv_user_data->received_feedback= true；
}


// CANバスにメッセージが到着するたびに呼び出される void onCanMessage(const
CanMsg& msg) { for (auto odrive: odrives)
    for (auto odrive: odrives) { onReceive(msg, *odrive)；
    }
}


void setup() { Serial.begin(115200); // PC
    側でシリアルポートが開くまで最大
    3秒待つ。


    // PC側でシリアルポートが開くまで最大3秒待つ。
    for (int i= 0; i< 30 && !Serial; ++i) { // PCが接続しない場合は
    、とにかく続行する。
        delay(100); }.
    }
    delay(200); Serial.println("Serial.println")
```

```
Serial.println("Starting ODriveCAN demo"); // ハートのコールバックを登録します。

// ハートビートとエンコーダのフィードバック・メッセージのコールバックを登録する
odrv0.onFeedback(onFeedback, &odrv0_user_data); odrv0.onStatus(onHeartbeat, &odrv0_user_data); odrv0.

// CANバス・インターフェースを設定し、初期化する。 この関数は、以下の関数に依存する。
// ハードウェアと使用しているCANスタック。
    Serial.println("CAN の初期化に失敗しました: リセットが必要です"); while
    (true); // 無限にスピンします。
}

Serial.println("Waiting for ODrive..."); while (!odrv0); // 無限
にスピンする } ; while (!odrv0); // 無限にスピンする } ;
while (!odrv0); // 無限にスピンする; while
(!odrv0_user_data.received_heartbeat) { }; // 無限にスピンする。
    pumpEvents(can_intf); delay(100) ;
}

Serial.println("found ODrive"); // バスの電圧と電流を要求する (1.0.1.1.1.1.1.1)

// バス電圧と電流を要求する（1秒のタイムアウト） Serial.println("tryinging to read bus voltage and
current"); Get_Bus_Voltage_Current_msg_t vbusGet_Bus_Voltage_Current_msg_t vbus ;
if (!odrv0.request(vbus, 1)) { Serial.println("vbus request
    failed!"); while (true); // 無限にスピンする。
}

Serial.print("DC voltage [V]: ");
Serial.println(vbus.Bus_Voltage); Serial.print("DC
current [A]: "); Serial.println(vbus.Bus_Current);
Serial.println("DC current [A]: ");
Serial.println(vbus.Bus_Current);
Serial.println(vbus.Bus_Current)

Serial.println(vbus.Bus_Current);Serial.println(vbus.Bus_Current);Serial
.println(vbus.; while (odrv0_user_data.last_heartbeat.Axis_State !
=Axis_State !
ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL) { odrv0.clearErrors() ;
    delay(1); odrv0.setState(ODriveAxisState::AXIS_STATE_CLOSED_LOOP_CONTROL); // 150msの間、イベントをポンプし
    ます。

    // この遅延は2つの理由から必要である; // 1.
    // 1.DC電源が不足するなどのエラー状態が発生した場合、ODriveは次のことを行うかもしれない。
```

```
    // DC 電源が不足するなどのエラー状態が発生した場合、ODrive は一時的に CLOSED_STOP に入ろうとします。  一時的にCLOSED_LOOP_CONTROL状態に入ろうとするので、 // ODriveの電源に頼ることはできない。
    // そのため、ODriveの電源に頼ることはできません。  が最初のハートビート応答を受信する可能性があります。
    //  ハートビート（デフォルト間隔100ms）。
    // バスが混雑している場合、setStateコマンドは通過しません。
    // 即座に、しかし遅らせることもできる。
        for (int i= 0; i< 15; ++i) { // 直ちに、しかし遅らせることができる。
          delay(10); pumpEvents(can_intf);
          }.
    }
  }

  Serial.println("ODrive running!"); }.
}

void loop() {
  pumpEvents(can_intf); // これは、いくつかのプラットフォームで、入力されるフィードバックCANメッセージを処理するために必要です。

  float SINE_PERIOD= 2.0f; // 位置コマンドサイン波の周���（秒） float t= 0.001 * millis(); float phase t * (TWO)

  float phase= t * (TWO_PI / SINE_PERIOD); // 位置指令サイン波の周期（秒） float t

  odrv0.setPosition( sin(phase), // 位置
    cos(phase) * (TWO_PI / SINE_PERIOD) // 速度フィードフォワード (オプション)
  ).

  シリアルプロッタ用に位置と速度を表示する if (odrv0_user_data.received_feedback) { // 位置と速度を表示する
    Get_Encoder_Estimates_msg_t feedback= odrv0_user_data.last_feedback; odrv0_user_data.received_feedback= false ；
    Serial.print("odrv0-pos:");
    Serial.print(feedback.Pos_Estimate); Serial.print(",");
    Serial.print("odrv0-vel:") ；
    Serial.println(feedback.Vel_Estimate) ；
  }
}
```

4.4.4 ROS SDK

以下の手順は、Ubuntu 23.04およびROS2 Ironでテストおよび検証されていますが、MACおよびWindowsプラットフォームではサポートされておらず、他のROS2リリースでは検証されていないため、使用する前に修正する必要があります。

4.4.4.1 odrive_canパッケージをインストールする

1. 新しいROS2ワークスペースを作成する (https://docs.ros.org/en/iron/index.html を参照)
2. git clone https://github.com/odriverobotics/odrive_can を使って、上記のワークスペース・ディレクトリの src ディレクトリにコードをダウンロードする。

```
colcon build --packages-select odrive_can
```

3. ターミナルでワークスペースのルート・ディレクトリに移動し、実行する：
4. 実行する前に環境を準備する：
5. ルーチン・ノードを実行する：

```
ソース/install/setup.bash
ros2 launch odrive_can example_launch.yaml
```

4.4.4.2 サービスの呼び出しとメッセージの表示

上記のodrive_can_nodeノードが名前空間odrive_axis0 (これは.
である。./launch/example_launch.yamlにあります）。4.4.4.1のノードが立ち上がると、以下のような公開されたトピックメッセージを見ることができるようになります：

```
ros2 トピック echo /odrive_axis0/controller_status ros2 トピック echo
/odrive_axis0/odrive_status
```

```
ros2 サービス コール /odrive_axis0/request_axis_state
ros2 サービスコール /odrive_can/srv/AxisState "{axis_requested_state: 4}"
```

を呼び出し、オープン・サービス・インターフェイスを呼び出します：

5 よくある質問と例外コード（更新予定）

5.1 よくある質問（FAQ）

5.2 例外コード

エラーカテゴリー	エラーコード	odrivetoolの表示	説明
システム例外	0x00000002	DC_BUS_UNDER_VOLTAGE	電源電圧が低すぎる
	0x00000004	DC_BUS_OVER_VOLTAGE	電源電圧が高すぎる
	0x00000008	DC_BUS_OVER_REGEN_CURRENT	電源逆（充電）電流が高すぎる高
	0x00000010	dc_bus_over_current	電源順方向（放電）電流オーバー高
ドライバ異常	0x00000001	INVALID_STATE	ドライブ状態異常
	0x00000040	MOTOR_FAILED	モーター故障
	0x00000100	エンコーダ異常	エンコーダー異常
	0x00000200	コントローラ異常	コントローラー異常
	0x00001000	MIN_ENDSTOP_PRESSED	下限トリガー
	0x00002000	最大エンドストップ	上限トリガー
	0x00004000	エストップ要求	緊急停止
	0x00020000	ホーミング_エンドストップなし	ゼロに戻るがリミットスイッチなし
	0x00080000	未知位置	位置情報がない
モータ異常	0x00000001	位相抵抗範囲外	相間抵抗が正常範囲を超えている
	0x00000002	位相インダクタンス範囲外	相間インダクタンスが🎧通常範囲を超える
	0x00000010	制御_期限切れ	FOC 周波数が高すぎる
	0x00000080	変調度	SVM変調異常
	0x00000400	電流センス飽和	相電流飽和
	0x00001000	電流制限違反	過大なモーター電流
	0x00020000	モータ温度超過	モーターの温度が高すぎる
	0x00040000	フェットサーミスタ温度超過	ドライブの温度が高すぎる
	0x00080000	タイマー更新ミス	FOC処理がタイムリーでない
	0x00100000	電流測定不能	相電流サンプリングが失われた
	0x00100000 CURRENT_MEASUREMENT_UNAVAILABLE	コントローラ故障	コントローラー故障
	0x00400000	i_bus_out_of_range	バス電流オーバー
	0x00800000	ブレーキ抵抗ディスアーム	ブレーキ抵抗駆動異常
	0x01000000	システム・レベル	システムレベル例外
	0x02000000	BAD_TIMING	相電流サンプリングのタイミング不良
	0x04000000	未知位相推定	モーター位置不明

	0x08000000	未知位相レベル	モーター速度不明
	0x10000000	不明なトルク	トルク不明
	0x20000000	unknown_current_command	トルク制御不明
	0x40000000	unknown_current_measurement	現在のサンプリング値 不明
	0x80000000	不明_vbus_voltage	電圧サンプリング値不明
	0x100000000	未知_電圧コマンド	電圧制御不明
	0x200000000	不明	電流ループゲイン不明
	0x400000000	コントローラ初期化中	コントローラ初期化例外
	0x800000000	アンバランス相	三相アンバランス
制御異常	0x00000001	オーバースピード	オーバースピード
	0x00000002	無効入力モード	制御入力モード不正
	0x00000004 UNSTABLE_GAIN	UNSTABLE_GAIN	位相ロックループのゲインが不安定
	0x00000020	無効推定値	位置/速度の不安定
	0x00000080	スピンアウト検出	機械的パワーと電気的パワーの不一致（エンコーダのキャリブレーションが正しくない、またはマグネットが不安定 または磁石の不安定性）
エンコーダ異常	0x00000001	UNSTABLE_GAIN	エンコーダの帯域幅が高すぎる
	0x00000002	cpr_polepaire_mismatch	CPRとポールペア番号の不一致
	0x00000004	応答なし	エンコーダーが応答しない
	0x00000400	秒エンコーダ通信エラー	2番目のエンコーダ通信エラー